IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 10 March 2026, accepted 14 March 2026, date of publication 19 March 2026, date of current version 25 March 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3675864

RESEARCH ARTICLE

Adversarial-Training-Based Robust Reinforcement Learning Control for Autonomous Vehicle Trajectory Tracking

DONGHYUN LEE[1], TAEYOUNG OH[1], JINKYEOM KIM[1], AND JINWOO YOO[2], (Member, IEEE)

[1]Graduate School of Automobile and Mobility, Kookmin University, Seoul 02707, Republic of Korea
[2]Department of Automobile and IT Convergence, Kookmin University, Seoul 02707, Republic of Korea

Corresponding author: Jinwoo Yoo (jwyoo@kookmin.ac.kr)

This work was supported by the National Research Foundation of Korea (NRF) grant funded by Korean Government [Ministry of Science and ICT (MSIT)] under Grant NRF-RS-2021-NR060086.

ABSTRACT Autonomous vehicles must maintain stable and accurate trajectory and speed tracking even in scenarios involving unpredictable disturbances and changing environmental conditions. Reinforcement learning (RL)–based controllers that directly output control inputs often exhibit performance degradation or instability when encountering out-of-distribution scenarios or disturbances. Model-based controllers are sensitive to model mismatch and parameter variations. Notably, even residual RL approaches offer limited robustness if disturbances are insufficiently represented during training. To address these challenges, this paper proposes an integrated tracking control architecture in which a linear quadratic regulator (LQR) serves as the nominal controller, while an RL policy provides residual compensation for steering and longitudinal (acceleration/deceleration) commands. To improve controller robustness, the training process is formulated as a min–max adversarial learning problem: The adversary learns a disturbance-injection policy while the controller minimizes tracking errors and ride-comfort penalties. Evaluation of the proposed method across a range of driving scenarios in the CARLA simulation environment demonstrates that it reduces trajectory errors and constraint violations compared with pure LQR, model predictive control, and an existing RL-based PID controller, both in the presence and absence of disturbances. These results demonstrate the improved tracking stability and robustness of the proposed controller.

INDEX TERMS Autonomous vehicle, reinforcement learning (RL), vehicle control, adversarial reinforcement learning.

I. INTRODUCTION

Trajectory tracking is an essential function of autonomous driving systems. It is directly related not only to safety—as it helps reduce lane departures and collision risks—but also to overall vehicle stability [1]. The objective of tracking control is to rapidly reduce tracking errors with respect to the reference path and speed while suppressing abrupt changes in steering and longitudinal (acceleration/deceleration) inputs, thereby generating continuous and smooth control commands [2]. In practice, high tracking accuracy and smooth driving behavior may impose competing requirements, and a controller capable of simultaneously satisfying both is required. Thus, achieving stable, precise tracking while maintaining ride comfort in real-world road environments is crucial for the practical deployment of autonomous driving technologies [3].

Notably, unlike the idealized assumptions adopted in modeling, real-world road environments involve time-varying uncertainties such as changes in the road–tire friction coefficient, longitudinal and lateral wind disturbances, sensor noise, observation errors, and control delays. These factors can introduce model mismatch and parameter variations, thereby degrading tracking performance [4], [5].

The associate editor coordinating the review of this manuscript and approving it for publication was Wen Chen.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

Consequently, for a given controller, even minor changes in operating conditions may lead to significant performance deterioration, making it difficult to simultaneously satisfy safety and driving-stability constraints [6]. Because safety is paramount in autonomous driving, controllers must remain stable and reliable not only under nominal conditions but also under adverse conditions [7].

To satisfy these requirements, two aspects must be jointly considered. First, a stable control architecture is needed to ensure predictable behavior within a computational budget that enables real-time implementation [8]. Second, the system must be able to adapt autonomously to time-varying factors, such as changes in road friction, disturbances, and delays [9], which are difficult to model. In other words, there exists a critical need for a framework that combines the stability and interpretability of model-based control with the data-driven adaptability of learning-based approaches.

Considering these aspects, this paper proposes an integrated framework that combines residual reinforcement learning (RL) with a linear quadratic regulator (LQR)-based tracking control structure, while enhancing robustness via a min–max formulation that injects adversarial disturbances during training. The LQR provides nominal longitudinal and lateral tracking commands and serves as a stabilizing nominal backbone in the intended operating regime. The RL policy learns bounded corrective residual inputs on top of the LQR outputs as an additive refinement, enabling adaptation to unmodeled dynamics and environmental variations without replacing the baseline controller. In addition, an adversary is introduced during training to generate disturbances within prescribed bounds to degrade the training environment. The driving policy is optimized to maintain tracking performance under such conditions, thereby improving resilience against worst-case scenarios.

The contributions of this work can be summarized as follows. First, we present an integrated control architecture in which RL learns residual inputs on top of a stable LQR baseline controller. Leveraging a unified longitudinal–lateral control structure, the proposed framework enables simultaneous longitudinal and lateral control within a single formulation, while achieving both stability and adaptability with real-time deployment capabilities. Second, controller robustness is enhanced through a min–max training strategy with adversarial disturbances, which allows it to preserve tracking performance and smooth driving behavior even under highly uncertain conditions. Third, a prior study [10] addressed the trade-off between tracking-error and ride-comfort by adaptively tuning the weights of a combined Pure Pursuit (PP) and proportional–integral–derivative (PID) control structure using RL. However, it did not explicitly incorporate a vehicle-dynamics-based model into the controller design, and diverse worst-case disturbances were not integrated into the training process. In contrast, the proposed method employs an LQR designed using vehicle dynamics as the nominal controller, with RL providing residual compensation for steering and longitudinal (acceleration/deceleration) commands. Moreover, by adversarially injecting disturbances at the action, dynamics, and observation levels during training, the proposed framework simultaneously strengthens tracking performance and driving stability under a wide range of uncertainties.

The remaining paper is organized as follows. Section II presents a review of prior research on conventional control methods and learning-based controllers and highlights key issues from the perspectives of uncertainty and robustness. Section III outlines the proposed RL strategy, including the design of the RL formulation. Section IV describes the proposed control method. Section V introduces the experimental setup and driving scenarios and discusses the performance evaluation results. Section VI presents the concluding remarks and outlines directions for future work.

II. RELATED WORK

Research on path-tracking and speed-tracking controllers can be categorized into three broad groups: model-free controllers, model-based controllers, and RL-based controllers.

A. MODEL-FREE CONTROLLERS

Model-free controllers compute control inputs by estimating the relationships between system states, inputs, and responses in a data-driven manner, i.e., using sensor measurements and simulation data, rather than relying on an explicit physical model of vehicle dynamics such as mass, cornering stiffness, or tire models [11], [12].

A representative example of a model-free controller is the PID controller. A key advantage of model-free approaches is that they can generate control inputs based on measurable errors and data without requiring an accurate vehicle dynamics model [13], [14]. Unknown disturbances and modeling errors can be compensated through error- and data-driven correction signals, reducing dependence on kinematic or dynamic vehicle models [15]. Even when the model features vary over time, e.g., owing to changes in tire characteristics and road friction, error-based feedback control can be applied directly. Moreover, because such controllers are simple to implement and computationally lightweight, they are well suited for real-time applications [16], [17].

Despite these advances, model-free controllers require extensive condition-specific tuning and are vulnerable to measurement noise. In autonomous driving tasks, where predictive capability is crucial—such as for accommodating changes in road curvature or deceleration of a leading vehicle—PID control is fundamentally limited because it relies primarily on instantaneous error feedback [18], [19].

B. MODEL-BASED CONTROLLERS

Model-based controllers compute control inputs—such as steering angle and acceleration/deceleration—by establishing a mathematical model of vehicle motion and leveraging the model's predictions of future vehicle behavior [20].

The LQR controller offers fast computation, a simple structure, and high interpretability [21], [22]. However, because it is based on linearization, its performance may degrade in strongly nonlinear regimes or under varying friction conditions. In addition, it exhibits limited capability to explicitly handle constraints—such as saturation of steering and longitudinal inputs—as they cannot be directly incorporated into the standard optimization-problem formulation [23], [24].

Model predictive control (MPC) can systematically incorporate constraints, resulting in high consistency with practical vehicle operating conditions. Moreover, lateral and longitudinal control can be integrated within a single optimization problem [25], [26]. Its drawback is increased computational complexity, as the optimization problem must be solved at each control cycle. Moreover, MPC performance is highly sensitive to model mismatch and design parameters such as weighting matrices, prediction horizons, and constraint settings [27], [28].

C. REINFORCEMENT LEARNING-BASED CONTROLLERS

To address the limitations of both model-free and model-based controllers, the RL-based tracking controller proposed in [10] interprets PP as a feedforward steering component and PID as an error-feedback component, forming a combined PP–PID steering command. Specifically, the authors establish an RL_PP_PID architecture, in which RL adaptively adjusts the weights of the two controllers according to driving conditions.

RL_PP_PID preserves the simplicity and real-time feasibility of the PP–PID framework while achieving an adaptive trade-off between tracking accuracy and ride comfort through online tuning of the controller weights. By integrating a learning-based adaptation mechanism on top of a stable baseline controller (PP+PID), this approach mitigates performance degradation attributable to model mismatch and changes in driving conditions, while maintaining performance within safe operating bounds.

Nevertheless, many existing RL-based tracking controllers either produce control inputs directly via the learned policy [29], [30] or adjust parameters of a geometric/kinematic baseline controller [31], [32]. Under conditions involving notable disturbances and model mismatch, such approaches may exhibit vulnerability to out-of-distribution scenarios. Moreover, when training is conducted only under idealized conditions without disturbances, robustness cannot be guaranteed [33], [34].

In summary, model-free controllers are simple to implement and offer strong real-time performance but exhibit limitations in terms of weak prediction capability, extensive tuning, and noise sensitivity. Model-based controllers provide advantages in interpretability and constraint handling but may be sensitive to model mismatch, nonlinearities, and disturbances. RL-based control offers adaptability; however, ensuring robustness under out-of-distribution conditions and diverse disturbance scenarios remains challenging. Motivated by these observations, this study is aimed at establishing an effective controller that preserves a stable nominal control structure while systematically strengthening tracking performance under disturbances and uncertainty.

III. PROPOSED RL STRATEGIES

A. PROPOSED RL STRATEGIES

The framework presented in this study learns continuous residual inputs for steering and longitudinal acceleration on top of an LQR nominal controller. A min–max structure is adopted, in which disturbances are injected adversarially during training. Notably, under this setting, critic estimation bias and training instability may be exacerbated; therefore, we adopt TD3 as the base algorithm, which exhibits improved update stability through twin critics and delayed policy updates that mitigate value overestimation. Moreover, TD3 is sample-efficient owing to its off-policy learning strategy, enabling experiences generated under diverse adversarial rollouts to be reused via a replay buffer. This characteristic is particularly advantageous for learning under disturbance conditions [35], [36]. Accordingly, TD3 is used as the foundation for stably implementing the proposed residual and adversarial training strategy. Throughout this paper, t denotes the discrete control time with sampling period Δt, and k denotes the look-ahead index for the finite-horizon preview sequence.

Building on the TD3 algorithm (Section III-A) and min–max adversarial RL framework (Section III-B), this section outlines the proposed LQR–residual RL integrated control scheme and the adversary-based robust training strategy.

1) STATE SPACE/ACTION SPACE

In RL, the design of appropriate state and action spaces is crucial for ensuring convergence and training stability. Autonomous driving tasks such as lane keeping and speed tracking frequently involve disturbances, delays, and model mismatch. Thus, policies may become unstable when observations are incomplete or the representation is overly complex. Accordingly, the state and action spaces in this study are constructed to capture both the essential tracking-error states and anticipated trend of future errors, while maintaining consistency between the control inputs applied in the environment and the learning inputs used for policy optimization.

The observation o_t combines a basic state vector and finite-horizon error sequence. The basic state consists of the lateral tracking error e_y, heading error e_ψ, path curvature κ, vehicle speed υ, and error-threshold flag h, i.e., $[e_y, e_\psi, \kappa, \upsilon, h]$.

In addition, predicted error sequences over a prediction horizon $\mathcal{N}_{pred}$, are included, i.e., $\{e_y^{(k)}\}_{k=1}^{\mathcal{N}_{pred}}$ and $\{e_\psi^{(k)}\}_{k=1}^{\mathcal{N}_{pred}}$.

At each time step t, these sequences are generated online from the current vehicle state and the given reference polyline by advancing along the reference according to the traveled distance. We first find the nearest reference index i_t corresponding to the current vehicle position (x_t, y_t). For

each look-ahead step $k = 1, \ldots, \mathcal{N}_{pred}$ with time step Δt, we define the look-ahead time $t_k = k\Delta t$ and compute the traveled distance $s_k = \hat{v}t_k$, where $\hat{v} = max(v_t, v_{min}^{pred})$ is used to avoid degenerate previews at near-zero speed. Given the reference point spacing Δs_{ref}, the corresponding future reference index is selected by converting distance to an index offset using the floor operator,

$$\Delta i_k = \left\lfloor \frac{s_k}{\Delta s_{ref}} \right\rfloor, i_{t+k} = clip\left(i_t + i_k, 0, N_{ref} - 2\right) \quad (1)$$

where N_{ref} is the total number of reference points. The predicted lateral and heading errors $e_y^{(k)}$ and $e_\psi^{(k)}$ are then computed with respect to the reference segment at i_{t+k} (and its reference heading) using the same error definitions as in the tracking controller.

Moreover, the observation includes the previously applied steering residual $\Delta\delta_{t-1}^{app}$ and a corner-state identifier $m_t \in \{0, 1, 2\}$, enabling the policy to reflect distinct filtering strategies and dynamics across straight, corner-entry, and corner-exit segments. The observation o_t can thus be defined as follows:

$$o_t = \left[e_y, e_\psi, \kappa, \upsilon, h, e_y^{(k)}{}_{k=1}^{\mathcal{N}_{pred}}, e_\psi^{(k)}{}_{k=1}^{\mathcal{N}_{pred}}, m_t\right] \quad (2)$$

Here, $\mathcal{N}_{pred}$ is the preview horizon length, and $\left\{e_y^{(k)}(t), e_\psi^{(k)}(t)\right\}_{k=1}^{\mathcal{N}_{pred}}$ denote the finite-horizon predicted error sequence appended to the basic state.

h is a binary flag indicating whether the current tracking error exceeds a predefined threshold, and m_t is a discrete segment indicator (e.g., straight/corner-entry/corner-exit).

The protagonist's action is designed in a residual form and added to the nominal LQR control input. This two-dimensional continuous action is defined as follows:

$$a_t = \left[\Delta\delta_t, \Delta a_{x,t}\right] \quad (3)$$

where $\Delta\delta_t$ denotes the steering residual, and $\Delta a_{x,t}$ denotes the longitudinal acceleration residual. The adversary's action $a_t^{\text{adv}} \in \mathbb{R}^{12}$ models disturbances that can degrade driving performance across three layers: action, dynamics, and observation. Specifically, at the action level, the adversary outputs $\Delta a_{x,t}^{\text{adv}}$ and $\Delta\delta_t^{\text{adv}}$, which are directly added to the protagonist's residual inputs. At the dynamics level, it outputs a longitudinal-acceleration bias $b_{a_x,t}$, lateral steering bias $b_{\delta,t}$, and road-friction scaling factor μ_t, thereby altering the vehicle's acceleration/steering responses and performance limits. Lastly, at the observation level, it outputs biases for lateral error, heading error, and speed $\left(b_{e_y,t}, b_{e_\psi,t}, b_{v,t}\right)$; observation-noise standard deviations $\left(\sigma_{e_y,t}, \sigma_{e_\psi,t}, \sigma_{v,t}\right)$; and an observation delay L_t(in steps), emulating sensor uncertainty as well as communication/processing delays. Then, the adversary action is defined as follows:

$$a_t^{adv} = \Big[\Delta a_{x,t}^{adv}, \Delta\delta_t^{adv}, C, b_{\delta,t}, \mu_t, b_{e_y,t}, b_{e_\psi,t}, b_{v,t}, \sigma_{e_y,t}, \sigma_{e_\psi,t}, \sigma_{v,t}, L_t\Big] \quad (4)$$

The disturbance components in (4) are injected into the closed-loop pipeline as follows. Action-level perturbations are added to the control command after computing the nominal controller and the residual policy:

$$u_t = \begin{bmatrix}\delta_t \\ a_{x,t}\end{bmatrix} = u_t^{LQR} + a_t^{RL} + \begin{bmatrix}\Delta\delta_t^{adv} \\ \Delta a_{x,t}^{adv}\end{bmatrix} \quad (5)$$

Dynamics-level perturbations are implemented as actuator-channel biases applied to the commanded inputs before sending them to CARLA:

$$\delta_t^{eff} = \delta_t + b_{\delta,t}, a_{x,t}^{eff} = a_{x,t} + b_{a_{x,t}} \quad (6)$$

Observation-level perturbations are applied to the measured tracking states by introducing an L_t-step delay and then adding bias and noise:

$$o_t = h(x_{t-L_t}) + b_t + \epsilon_t, \epsilon_t \sim \mathcal{N}(0, \sum_t)) \quad (7)$$

where $b_t = [b_{e_y,t}, b_{e_\psi,t}, b_{v,t}]^T$ and $\sum_t$ is diagonal with $[\sigma_{e_y,t}, \sigma_{e_\psi,t}, \sigma_{v,t}]$.

L_t denotes the observation delay measured in discrete time steps, and $\sigma_{\cdot,t}$ represents the standard deviation of the corresponding zero-mean observation noise.

Such a multi-level disturbance model encourages the learned policy to operate robustly under a broad range of uncertainty conditions compared with single-disturbance settings. Table 1 summarizes the upper bounds of each disturbance, designed to avoid non-physical values.

TABLE 1. Upper limit of each disturbance.

Disturbance	Upper limit	Unit
$Act_{adv}\, a_x$	1.5	m/s^2
$Act_{adv}\delta$	0.1	rad
$Dyn_{lon}bias$	1.5	m/s^2
$Dyn_{lat}bias$	0.1	rad
$Dyn\ friction\ scale$	[0.4 1]	-
$Obs_{bias}e_y$	0.5	m
$Obs_{bias}e_{psi}$	0.1	rad
$Obs_{bias}e_v$	1.5	m/s
$Obs_{noise}e_y$	0.3	m
$Obs_{noise}e_{psi}$	0.05	rad
$Obs_{noise}e_v$	0.5	m/s
$Obs_{latency}$	12	steps

Our method is conceptually related to perturbation-injection approaches such as noise injection, domain randomization, and adversarial perturbation training, in that robustness is improved by training under perturbed execution. The key difference lies in where perturbations are applied and how they are budgeted and optimized. Rather than perturbing only the observation or action space, we define and inject perturbations across the observation channel, the control-input (residual) channel, and the vehicle-dynamics channel, each constrained by an explicit bounded disturbance budget.

Training can be interpreted as an approximate minimax optimization over a product disturbance set $\mathcal{D} = \mathcal{D}_1 \times \cdots \times \mathcal{D}_k$ with channel-wise bounds $\|d_i\|_\infty \leq \varepsilon_i$, enabling systematic comparisons under matched disturbance strengths.

Compared to purely random perturbations, a learned adversary concentrates training on high-impact (hard) disturbance directions within the same budget, which can improve worst-case robustness. In addition, the LQR-backbone plus RL-residual structure introduces a stabilizing inductive bias: the learned policy focuses on bounded corrective residuals rather than re-learning the entire controller from scratch, which can reduce function complexity and mitigate catastrophic degradation under perturbations.

Adversarial training may overfit to the particular disturbance parameterization and strategies discovered by the adversary, potentially leaving gaps for unseen perturbations. To mitigate this risk, we adopt a multi-channel disturbance set that spans observation, actuation/control-input, and dynamics factors, and we train under explicit bounded disturbance budgets with optionally scheduled strengths to avoid reliance on a single disturbance pattern. Nevertheless, robustness is fundamentally defined with respect to the chosen disturbance set D, and perturbations outside D are not guaranteed. As a promising direction for future work, D can be evaluated and expanded to include additional unseen disturbance families, and mixtures or ensembles of adversaries can be considered to further improve generalization.

2) REWARD DESIGN

In RL-based lane-keeping control, the reward design should not only minimize tracking errors but also simultaneously satisfy driving stability, ride comfort, and safety constraints. In this study, lateral tracking error, heading error, and longitudinal speed error are set as primary objectives for the lane-keeping task. In addition, a prediction-based error term and a curvature–attitude coupled term are incorporated to enhance stable tracking performance in curved segments. To suppress excessive variations in steering and acceleration, which can degrade driving quality, both control-effort and steering-rate penalties are incorporated.

In Eq. (8)–(23), all weights $w_\cdot$ are nonnegative scalars, and the thresholds (e.g., x_{gate}, e_y^{tol}, e_ψ^{tol}, j_{lat}^{tol}, j_{lon}^{tol}) specify the activation points of gating or penalty terms.

a: TRACKING-CENTERED BASELINE REWARD

Let the error states at time t be the lateral error $e_y(t)$, heading error $e_\psi(t)$, and speed error $e_v(t)$. The baseline tracking reward is designed in the form of the following squared-penalty term:

$$r_{track}\,(t) = -w_{e_y} e_y\,(t)^2 - w_{e_\psi} e_\psi\,(t)^2 - w_{e_v} e_v\,(t)^2 \tag{8}$$

To modulate the sensitivity of the error penalties according to vehicle speed and path curvature, we adaptively vary the corresponding weights. Specifically, higher speeds impose a stronger penalty on lateral error, while larger curvature imposes a stronger penalty on heading error, as follows:

$$\begin{aligned} w_{e_y}^{eff} &= w_{e_y}\,(1 + \alpha_v max\,(0, v\,(t)))\,, \\ w_{e_\psi}^{eff} &= w_{e_\psi}\,(1 + \alpha_k\,|K\,(t)|) \end{aligned} \tag{9}$$

This design enables stable tracking in straight segments without imposing excessive constraints, while enforcing stronger error suppression in curved and high-speed regimes.

b: PROGRESS- AND DISTANCE-BASED REWARD

Minimizing tracking error alone can lead to convergence to a stationary state. To address this and encourage forward motion, a distance-based reward is introduced. The incremental forward distance is defined as in (10), and it is converted into a reward term as in (11).

$$\Delta s\,(t) = max\,\left(0, v\,(t)\,cos\,\left(e_\psi\,(t)\right)\right)\Delta t \tag{10}$$

$$r_{dist}\,(\mathrm{t}) = w_{dist}\,\Delta s\,(t)\,g\,(t) \tag{11}$$

where $g(t)$ is a soft gating term introduced to prevent exploitation of the forward-progress reward in regimes with large tracking errors and/or large speed errors, defined as

$$g\,(t) = g_{e_y}\,(t)\,g_{e_\psi}\,(t)\,g_{e_v}\,(t)\,,\, g_x\,(t) = \frac{1}{1 + \left(\frac{|x(t)|}{x_{gate}}\right)^p} \tag{12}$$

In other words, forward-progress reward is designed to be naturally attenuated as tracking performance degrades.

c: COST OF CONTROL INPUT AND STEERING RATE OF CHANGE

To limit excessive control inputs, the reward for the steering and longitudinal-acceleration inputs is defined as follows:

$$r_{ctrl}\,(\mathrm{t}) = -w_\delta \delta\,(t)^2 - w_{a_x} a_x\,(t)^2 \tag{13}$$

In addition, because abrupt changes in the steering input can degrade ride comfort and induce unstable yaw behavior, the following penalty is imposed on the steering rate of change $\dot{\delta}$:

$$\dot{\delta}\,(\mathrm{t}) \approx \frac{\delta\,(t) - \delta\,(t - \delta t)}{\Delta t},\, r_{smooth}\,(t) = -w_{\dot{\delta}} \dot{\delta}\,(t)^2 \tag{14}$$

d: PREDICTION-BASED ERROR AND MAXIMUM PREDICTED-ERROR PENALTY

The environment state includes not only the current state but also the predicted error sequences over the next N_{pred} steps, $\left\{e_y^{(k)}(t)\right\}$ and $\left\{e_\psi^{(k)}(t)\right\}$. This information is incorporated into the reward to proactively reduce future errors. Both the mean absolute error and maximum absolute error are used.

$$\begin{aligned} r_{pred}\,(t) = &-w_{p,y} \frac{1}{N} \sum_{k=1}^{N} \left| e_y^{(k)}\,(t) \right| \\ &- w_{p,\psi} \frac{1}{N} \sum_{k=1}^{N} \left| e_\psi^{(k)}\,(t) \right| \\ &- w_{p,max} \left(\max_k \left| e_y^{(k)}\,(t) \right| + \max_k \left| e_\psi^{(k)}\,(t) \right| \right) \end{aligned} \tag{15}$$

The mean term reflects the overall future tracking performance, whereas the maximum term serves to suppress large deviations that may occur at specific time instants.

e: CURVATURE–HEADING COUPLING TERM

To prevent large heading errors in high-curvature segments from leading to abrupt trajectory deviations, a curvature–heading error coupling penalty introduced:

$$r_{curv}(t) = -w_c |K(t)| \left|e_\psi(t)\right| \tag{16}$$

f: CONSTRAINT-BASED PENALTY AND SAFETY FLAG

To ensure training stability and safety, three constraint indicators c_1, c_2, and c_3 are defined and incorporated into the reward:

1) c_1 : Corridor constraint — increases in a hinge form when the allowable error bounds $\left(e_y^{\text{tol}} \mid e_\psi^{\text{tol}}\right)$ are exceeded.

$$c_1(t) = \left[\left|e_y(t)\right| - e_y^{tol}\right]_+^2 + \left[\left|e_\psi(t)\right| - e_\psi^{tol}\right]_+^2 \tag{17}$$

2) c_2 : Jerk constraint — when the lateral/longitudinal jerk exceeds a threshold, the amount of violation is accumulated as a penalty.

$$c_2(t) = \left[|j_{lat}(t)| - j_{lat}^{tol}\right]_+^p + \left[|j_{lon}(t)| - j_{lon}^{tol}\right]_+^p \tag{18}$$

3) c_3 : ISS/ultimate-bound–type stability constraint — imposes a penalty when the increment of the normalized Lyapunov candidate V fails to satisfy a specified condition.

$$V(t) = x(t)^T P x(t), x(t) = \left[e_y, v_y, e_\psi, e_r, e_v, e_{y,int}\right]^T \tag{19}$$

$$c_3(t) = \left[\Delta V(t) + \eta \|x\|^2 - \beta\right]_+^p,$$
$$\Delta V(t) = V(t) - V(t - \Delta t) \tag{20}$$

Here, $[x]_+ = \max(0, x)$ denotes a hinge operator that penalizes only constraint violations. In the actual reward, the corridor constraint is weighted most strongly, while the jerk penalty is gated to prevent it from dominating in segments where tracking performance degrades considerably. Then, the constraint penalty included in the reward can be defined as follows:

$$r_{cons}(t) = -w_{cor} c_1(t) - w_{jerk} c_2(t) - w_{iss} c_3(t) \tag{21}$$

g: TERMINAL REWARD AND ADDITIONAL TERMS

A large negative terminal penalty is assigned when a collision or off-road event occurs, whereas a positive terminal bonus is assigned upon successful completion of the goal:

$$r_{term}(t) = \begin{cases} -R_{crash}, & collidedoroffroad \\ +R_{succ}, & reach\ edgoal \\ 0, & otherwise \end{cases} \tag{22}$$

An additional bonus is provided when the system remains stable within a small-error region, thereby reinforcing precise tracking performance.

h: FINAL REWARD FUNCTION

The overall reward at time t can be expressed as follows:

$$r(t) = r_{track}(t) + r_{dist}(t) + r_{ctrl}(t) + r_{smooth}(t) + r_{pred}(t) + r_{curv}(t) + r_{cons}(t) + r_{term}(t) \tag{23}$$

This reward function encourages the minimization of tracking errors, while ensuring forward progress, improving ride comfort and control stability, and suppressing constraint violations.

B. TD3 ALGORITHM AND ADVERSARY RL OVERVIEW

1) TD3 BACKBONE

Twin delayed deep deterministic policy gradient (TD3) is an off-policy actor–critic RL algorithm designed for control problems with continuous action spaces. The algorithm was designed to mitigate the training instability, caused by overestimation of the value function, commonly encountered in deep deterministic policy gradient methods [38]. TD3 consists of an actor that outputs a deterministic action given the current state and a critic that approximates the expected cumulative return over state–action pairs. Training stability is further improved through experience replay using a replay buffer, as well as the use of target networks [39]. The network architecture of TD3 is illustrated in Fig. 1.

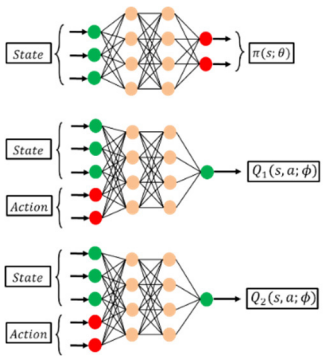


FIGURE 1. TD3 algorithm network architecture.

Adversarial RL is a training methodology that aims to enhance policy robustness by forcing an agent to interact with intentionally designed adversarial factors during training. This approach exposes the agent to non-ideal conditions such as model uncertainty, disturbances, and observation noise. Conventional RL typically assumes a fixed and idealized environment, and the learned policies are therefore vulnerable to distribution shifts between training and deployment. In contrast, adversarial RL aims to improve both generalization performance and safety by proactively exposing the policy to worst-case conditions from the training stage onward [40].

In general, adversarial RL is formulated as a two-player Markov game, or equivalently, a min–max optimization problem in which a protagonist and an adversary interact [41]. Let the protagonist policy π_θ and adversary policy π_ψ^a select actions a^p and a^a, respectively. Denoting the expected return by $J(\theta, \psi)$, the objective can be expressed as follows:

$$\min_\theta \max_\psi J(\theta, \psi) = \min_\theta \max_\psi \mathbb{E}_{\pi_\theta, \pi_\psi^a}[r(s, a^p, a^a)] \quad (24)$$

Here, s denotes the state, a^p and a^a denote the protagonist and adversary actions, respectively, and $r(s, a^p, a^a)$ is the instantaneous reward. $\mathbb{E}_{\pi_\theta, \pi_\psi}[\cdot]$ denotes the expectation under the protagonist policy π_θ and adversary policy π_ψ.

Adversarial RL frameworks have been actively adopted in safety- and reliability-critical applications, such as autonomous driving and robotic control. For instance, He et al. [39] proposed an adversarial RL approach for complex traffic scenarios such as highway merging, in which the behaviors of surrounding vehicles, uncertainties in traffic flow, and observation noise are modeled adversarially to learn a robust decision-making policy that explicitly accounts for safety constraints like merge success and collision avoidance.

2) INTEGRATION OF PROTAGONIST AND ADVERSARY AGENTS

This section describes the min–max adversarial learning framework that combines a protagonist (main) agent (to improve path/speed tracking performance) and an adversary agent (to perturb driving conditions to induce worst-case scenarios) within the same environment. The integrated architecture is characterized by three features: (a) a disturbance-injection interface, (b) consistency in applying mixed actions to the real system, and (c) simultaneous training based on opposing objective functions.

a: DISTURBANCE INJECTION AND MIXED-ACTION GENERATION

At each time step t, the main agent receives the state o_t defined in Section III-A1 as the input and outputs a residual control input, defined as

$$a_t^{RL} = \begin{bmatrix} \Delta\delta_t \\ \Delta a_{x,t} \end{bmatrix} \in \mathbb{R}^2 \quad (25)$$

The adversary agent, based on the same observation, outputs a 12-dimensional disturbance vector $a_t^{\text{adv}} \in \mathbb{R}^{12}$, where the action-disturbance component is directly mixed with the main agent's residual action. Following Section III-A.1, the remaining 10 dimensions are mapped into longitudinal/lateral dynamics biases, friction coefficients, and observation bias/noise/delay. These parameters are then injected into the environment's dynamics and observation channels. The action-disturbance component is defined as follows:

$$\tilde{a}_t^{adv} = \begin{bmatrix} \Delta\delta_t^{adv} \\ \Delta a_{x,t}^{adv} \end{bmatrix} \quad (26)$$

Coupling at the action level is achieved using a mixing coefficient $\eta_t \in [0, 1]$, yielding the following raw mixed residual.

$$a_t^{mix,raw} = (1 - \eta_t)\, a_t^{RL} + \eta_t \tilde{a}_t^{adv} \in R^2 \quad (27)$$

In the initial stage of training, η_t is set to a small value to allow the main agent to adapt to the baseline tracking task. As training progresses, η_t is gradually increased, encouraging the policy to become robust under increasingly severe disturbances.

b: APPLIED RESIDUAL AND LQR-BASED INTEGRATED CONTROL

Before being combined with the baseline controller output (LQR), the residual is transformed into the actually applied residual action, $a_t^{mix,app}$, through channel-wise scaling and filtering for the steering and acceleration channels.

First, the raw mixed residual in (27) is defined as follows:

$$a_t^{mix,raw} = \begin{bmatrix} d_t^{raw} \\ a_{x,t}^{raw} \end{bmatrix} \quad (28)$$

For updating, the steering residual is passed through a low-pass filter, depending on the previously applied value d_{t-1}^{app} and current road-segment mode $m_t \in \{0, 1, 2\}$ (straight / corner entry / corner exit), as follows:

$$d_t^{app} = \alpha(m_t)\, d_{t-1}^{app} + (1 - \alpha(m_t))\, k_\delta\, clip\left(d_t^{raw}\right) \quad (29)$$

Here, $\alpha(m_t)$ is smaller on straight segments and larger in corner-entry and corner-exit segments to prevent abrupt changes in residual steering. k_δ is a scaling factor for the residual steering, and the $clip(\cdot)$ operation saturates the command within the physically allowable range. For the longitudinal-acceleration residual, a more gradual dynamic characteristic than steering is assumed; thus, only scaling and saturation are applied as

$$a_{x,t}^{app} = k_a clip\left(a_{x,t}^{raw}\right) \quad (30)$$

The applied residual vector is as defined in (31). The final control input is obtained by superposition with the LQR baseline control input $\left(\delta_t^{\text{LQR}} \mid a_{x,t}^{\text{LQR}}\right)$, which is then applied to the vehicle, as defined in (32).

$$a_t^{mix,app} = \begin{bmatrix} d_t^{app} \\ a_{x,t}^{app} \end{bmatrix} \quad (31)$$

$$\delta_t = \delta_t^{LQR} + d_t^{app},\ a_{x,t} = a_{x,t}^{LQR} + a_{x,t}^{app} \quad (32)$$

The value-function learning and policy optimization of the main agent are designed to be consistent with this actually applied residual representation. In the experience replay memory (replay buffer), the observation o_t, main agent's raw action a_t^{RL}, actually applied residual action $a_t^{\text{mix,app}}$ to the environment, and constraint indicators $c_1(t)$, $c_2(t)$, $c_3(t)$ are stored.

During the value-function update, $a_{t+1}^{\text{mix,app}}$ is reconstructed from the policy output at the next state, o_{t+1}; adversary action; and mixing coefficient η_{t+1}, following the same procedure as in (27) and (28)–(31). A distributional target is then

formed to train the quantile twin-Q. The distributional target is defined as follows:

$$y_t^{(n)} = r_t + \gamma\,(1-d_t)\,\min_i Z_{\phi,i}^{(n)}\left(o_{t+1}, a_{t+1}^{mix,app}\right) \tag{33}$$

Using the same residual representation in both the training and execution phases, we minimize the bias arising from representational mismatch between the policy outputs and value-function inputs.

c: MIN–MAX ADVERSARIAL LEARNING AND RISK-SENSITIVE OPTIMIZATION WITH CONSTRAINTS

The main agent maximizes an objective function that simultaneously accounts for CVaR-based tail-risk minimization and constraint satisfaction, using the quantile twin-Q critic and constraint indicators $c_j(t)$ defined in Sections III-A2. Specifically, at each time step, CVaR_α is computed as the average of the lower α portion of the distributional output $Z_{\phi,i}(o_t, a_t^{\mathrm{mix,app}})$. Then, using the hinge-type violations of the three constraint indicators, the following objective is formulated:

$$j(\theta) = \mathbb{E}\left[\begin{array}{l} CVaR_\alpha\left(\min_i Z_{\phi,i}\left(o_t, a_t^{mix,app}\right)\right) \\ -\sum_{j=1}^{3}\lambda_j\left[c_j\left(o_t, a_t^{mix,app}\right)-\varepsilon_j\right]_+ \end{array}\right] \tag{34}$$

where $a_t^{\mathrm{mix,app}}$ is the actually applied residual obtained by mixing and transforming the main agent's residual and adversary action according to (34), (42), and (44). λ_j and ε_j denote the dual variable and allowable level for constraint c_j, respectively. The policy parameters θ are updated to maximize $j(\theta)$, while the dual variables λ_j are updated via ascent, as in (35), based on the episode-wise average constraint violation.

$$\lambda_j \leftarrow \left[\lambda_j + \rho\left(\bar{c}_j - \varepsilon_j\right)\right]_{[0,\lambda_{max}]} \tag{35}$$

Conversely, the RL reward of the adversary agent, within the same environment, is equal to the negative cumulative return of the main agent. The TD3-based policy is trained in the direction that minimizes the main agent's performance. Specifically, by simultaneously adjusting action-, dynamics-, and observation-level disturbances, the adversary searches—under a min–max perspective—for disturbance patterns that degrade the main agent's CVaR performance.

C. RL NETWORK ARCHITECTURE

As mentioned previously, the protagonist agent adopts a TD3-based actor–critic architecture, and value-function estimation is extended to a distributional form to improve training stability and risk-sensitive performance. The actor $\pi_\theta(o_t)$ represents a deterministic policy that takes the observation o_t as input and outputs the residual control action $a_t^{RL} = [\Delta\delta_t, \Delta a_{x,t}]$. Unlike the standard TD3 implementation, which uses twin Q critics that produce scalar values $Q_{\phi,i}(o_t, a_t)\,(i \in \{1,2\})$, we design the critic as a quantile twin-Q network such that each critic outputs N quantiles and approximates a distributional value function $Z(o_t, a_t) \in \mathbb{R}^N$ [42], [43], expressed as

$$Z_{\phi,i}(o_t, a_t) = z_{\phi,i}^{(n)}(o_t, a_t)_{n=1}^{N},\ \mathrm{i} \in \{1,2\} \tag{36}$$

Here, $Z(o_t, a_t)$ denotes the conditional return distribution $\mathcal{L}(G_t|o_t, a_t)$ with $G_t = \sum_{k=0}^{\infty}\gamma^k r_{t+k}$. We use $\tau_n = \frac{n-\frac{1}{2}}{N}$ and interpret $z_{\phi,i}^{(n)}(o_t, a_t)$ as the τ_n-quantile estimate of $Z(o, a)$.

$$\hat{Q}_{\phi,i}(o_t, a_t) = \frac{1}{N}\sum_{n=1}^{N} z_{\phi,i}^{(n)}(o_t, a_t) \tag{37}$$

The scalar $\hat{Q}_{\phi,i}(o, a)$ corresponds to the mean of $Z(o, a)$.

Then, critic training is performed not with a simple mean-squared error (MSE) objective but with a (quantile-regression based) Huber-type loss. This choice is motivated by the fact that, in bootstrapped off-policy learning, the TD error can occasionally contain large-magnitude outliers, and such effects can be further amplified under the non-stationary return distributions induced by disturbance injection in adversarial training. Since MSE penalizes errors quadratically, rare but large TD errors may dominate the critic gradient and lead to unstable value updates. In contrast, the Huber loss behaves quadratically for small errors while transitioning to a linear regime for large errors, thereby limiting the influence of outliers and improving robustness of the estimator. This robustification principle has been widely adopted to reduce sensitivity to noisy/outlying samples in learning-based regression settings [44], and has also been used as a key ingredient for robustness against adversarial/Byzantine perturbations in distributed learning [45]. Accordingly, we employ the quantile Huber loss in Eq. (38) to stabilize critic learning under disturbance-driven non-stationarity, while retaining local accuracy around typical TD errors. The quantile Huber loss can be expressed as follows:

$$\mathcal{L}_{\mathcal{QR}}(\phi) = \frac{1}{NN_t}\sum_{n=1}^{N}\sum_{m=1}^{N_t}\rho_{\tau_n}^{\delta_H}\left(y_t^{(m)} - z_{\phi,i}^{(n)}(o_t, a_t)\right)$$
$$\tau_n = \frac{n-\frac{1}{2}}{N} \tag{38}$$

In Eq. (38), $\rho_\tau^{\delta_H}(\cdot)$ is the quantile Huber loss with Huber threshold $\delta_H > 0$, applied to the quantile residual between the target sample and the predicted quantile. The indices m and n enumerate the target and current quantile samples, respectively. $\delta_H > 0$ is the Huber threshold (smoothing parameter) used in the quantile Huber loss. Following TD3, we construct the distributional target using policy smoothing $a' = \pi_\theta - (o_{t+1}) + \epsilon$ and the double-critic operator:

$$Y_t = r_t + \gamma(1-d_t)\min_{i\in 1,2} Z_{\phi^-}\left(o_{t+1}, a'\right) \tag{39}$$

where the minimum is applied elementwise over quantiles, and $\{y_t^{(m)}\}_{m=1}^{N}$ are the resulting target quantile samples.

In addition, unlike standard TD3, we employ a risk-sensitive optimization objective for actor updates using

the conditional value at risk (CVaR_α), computed by aggregating the lower-tail portion of the distributional outputs $Z_{\phi,i}$. This enables the policy to account for tail risk as well as average performance.

In terms of the architecture, all networks adopt a multilayer perceptron (MLP) as the backbone. Orthogonal initialization is applied to all linear layers, and the output-layer gain is adjusted according to the objective. Thus, the proposed structure extends the scalar-Q, mean-optimization formulation of conventional TD3 by combining distributional value estimation with a risk-sensitive objective. The actor optimization objective, which maximizes CVaR_α while satisfying constraints, is defined as follows:

$$CV_aR_\alpha\,(Z) = \frac{1}{K}\sum_{n=1}^{K} z^{(n)}_{sorted},\ K = max\,(1, [\alpha N]) \tag{40}$$

Here, $\alpha \in (0, 1]$ is the CVaR tail fraction, and $[\alpha N]$ denotes the integer number of quantiles included in the tail averaging (e.g., rounding down to an integer).

$$\begin{aligned} J(\theta) &= \mathbb{E}[CV_aR_\alpha(\min_i Z_{\phi,i}(o_t, a_t)) \\ &\quad - \sum_{j=1}^{3} \lambda_j [c_j(o_t, a_t) - \varepsilon_j]_+] \\ \theta &\leftarrow arg\max_\theta J\,(\theta) \end{aligned} \tag{41}$$

In Eq. (41), the constraint set is indexed by $j \in \{1, \ldots, J$: $c_j(\cdot)$ denotes the j-indexed constraint function with allowable level ϵ_j, and $\lambda_j \geq 0$ is the associated dual weight; $[z]_+ = \max(0, z)$ is the hinge operator.

The adversary agent also adopts a TD3-based actor–critic architecture; however, to limit training and implementation complexity, its critic follows the standard TD3 formulation and uses scalar twin Q-functions. The adversary then generates a policy output $a_t^{\text{adv}} \in \mathbb{R}^{12}$.

The MSE loss function architecture for the TD3 critic and architecture of the quantile twin-Q network are shown in Figs. 2 and 3, respectively.

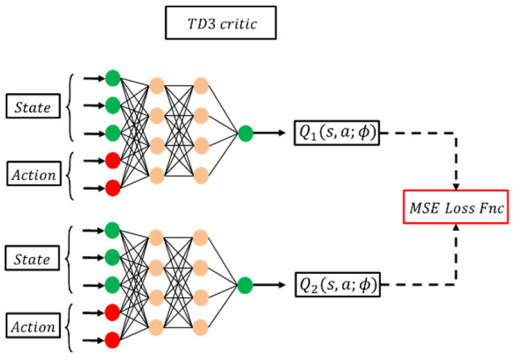


FIGURE 2. TD3 critic MSE loss Fnc architecture.

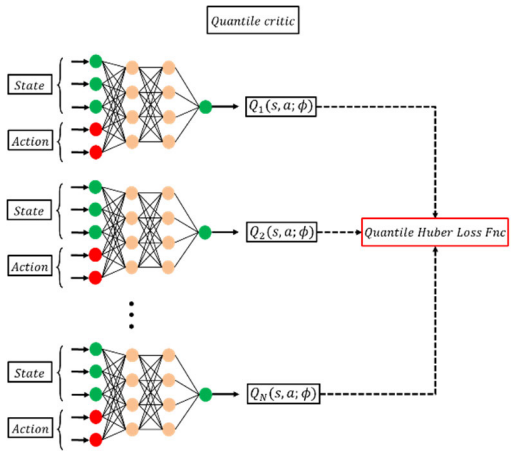


FIGURE 3. Quantile Twin-Q network architecture.

IV. PROPOSED ALGORITHM

Building on the framework outlined in Section III, this section describes the proposed learning-and-control algorithm. The architecture of the proposed algorithm is illustrated in Fig. 4.

Prior end-to-end formulations that directly output low-level control commands have reported practical limitations, including training instability and performance degradation in challenging regimes (e.g., high-curvature segments), as well as sensitivity to data coverage in rare corner cases and limited interpretability of model-free decision logic [46], [47]. These issues are particularly relevant to our setting, where adversarial disturbance injection and constrained actuation can induce non-stationary learning targets and occasional large deviations, and where the executed control may differ from the raw policy output due to post-processing (e.g., saturation, rate limits, filtering) and mixing. Motivated by these considerations, we adopt a residual policy on top of a nominal tracking controller. Residual RL has been shown to improve performance while maintaining safe and robust operation by learning corrective residual actions around a stabilizing base controller [48]. In our implementation, we further maintain action-semantics consistency by training the critic on the actually executed (post-processed/mixed) residual action, which aligns the learning objective with the closed-loop execution.

The upper Learning Layer performs min–max adversarial training between a distributional (quantile) TD3-based main agent and TD3-based adversary agent, while the lower Control Layer superimposes the learned residual policy onto an LQR-based integrated controller to generate the final control input. The final command corresponds to the acceleration and steering inputs applied to the vehicle. ''STW'' denotes the steering wheel angle.

A. LEARNING-CONTROL LOOP ARCHITECTURE

The environment and baseline-control layer generates a reference trajectory from the XODR-based route, a predefined speed profile, and the vehicle dynamics. It then computes

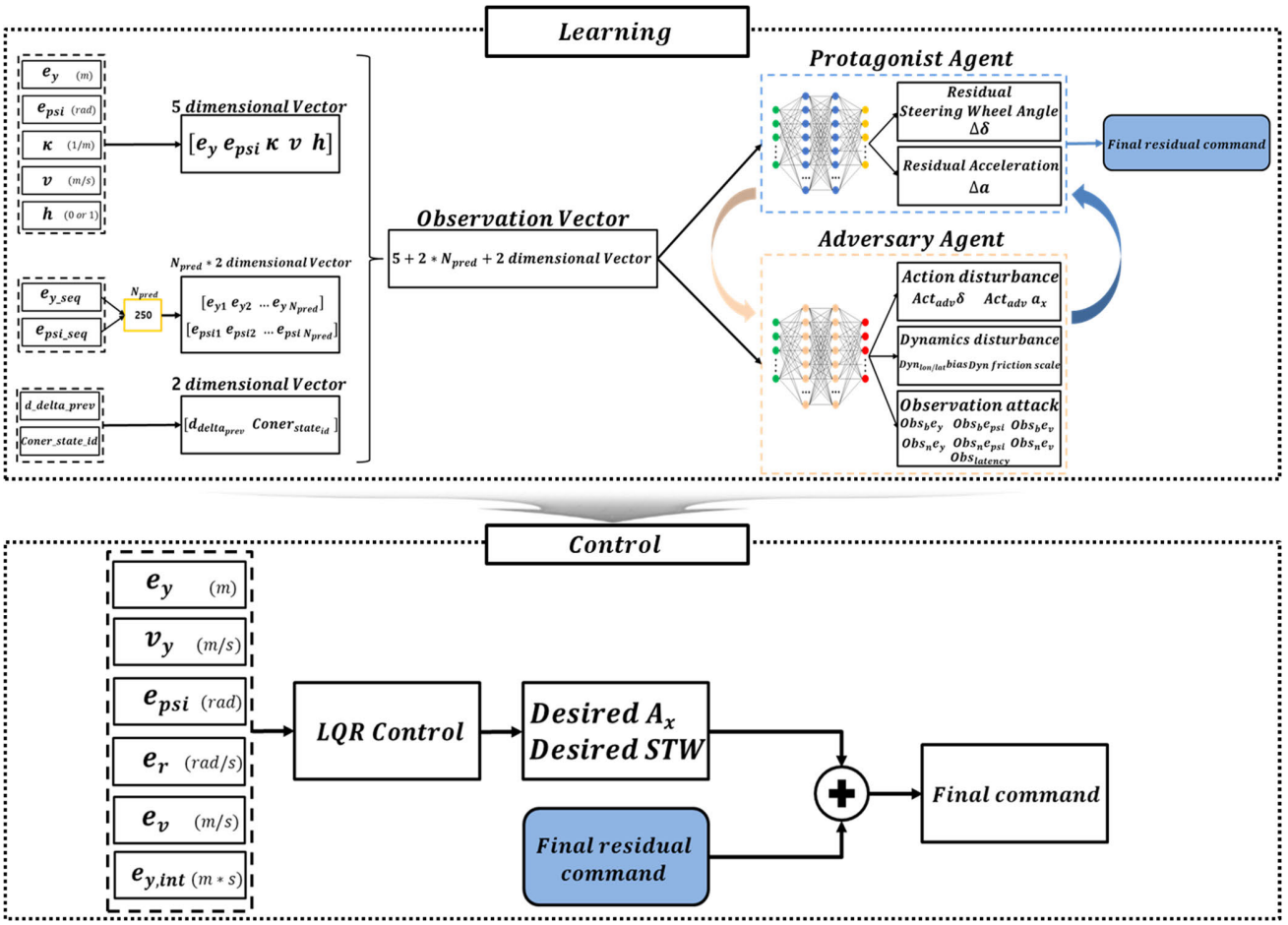


FIGURE 4. Architecture of the proposed algorithm.

the LQR baseline control inputs based on this reference. Up to this stage, the system operates in the same manner as a conventional model-based lane-keeping controller, and regardless of whether learning is enabled, it always provides stable baseline tracking performance.

In the observation-construction stage, the environment forms the observation o_t defined in Section III-A1 using the vehicle position, speed, attitude, and reference-trajectory information. This observation includes not only the current errors but also the predicted future error sequence, corner-segment classification information, and previously applied residual steering value, enabling the learning layer to simultaneously account for the upcoming trajectory and vehicle dynamics.

During interaction within the learning layer, the main agent takes o_t as input and outputs a residual control command, while the adversary agent, based on the same observation, produces disturbance parameters for the action, dynamics, and observation channels. The main agent compensates for the shortcomings of the LQR controller, whereas the adversary agent artificially generates worst-case conditions.

In the residual mixing and application stage, the main-agent residual and adversary residual are combined via a weighted sum using the mixing coefficient $\eta_t \in [0, 1]$. The resulting residual is then passed through rate-of-change constraints, saturation limits of the actual steering and acceleration commands, as well as segment-wise filtering for corner sections, to determine the final residual. This residual is superimposed on the LQR baseline input and applied to the vehicle, and the final control command is generated at the Control Layer. In Fig. 4, the Control-Layer state variables v_y, e_r, and $e_{y,\mathrm{int}}$ denote the lateral velocity, yaw-rate error, and integral of lateral error $e_{y,\mathrm{int}} = \int e_y \, dt$, respectively.

In the logging and experience accumulation stage, the environment returns not only the reward and constraint metrics, but also computed quantities such as the actually applied mixed residual, value of the Lyapunov candidate function, residual rate-of-change, and jerk-related indicators. This information is used for replay-memory sampling and stability monitoring during training.

With this structure, the action representation defined in the Learning Layer and input structure used in the actual Control Layer are maintained consistent at all times.

B. ADVERSARIAL EXPERIENCE GENERATION

The training phase of the proposed algorithm consists of offline episodic rollouts conducted in a simulator. Within each episode, the main and adversarial agents share the

same environment and generate experience in the following manner.

At the beginning of each episode, the vehicle state and reference trajectory are initialized, and the environment provides the initial observation o_0. As time progresses, the adversarial agent continuously outputs a 12-dimensional disturbance parameter—covering the three layers of actions, dynamics, and observations—based on the current observation.

During early training, both the allowable disturbance ranges and mixing coefficient η_t are set within a narrow range; they are then gradually expanded as training proceeds. In other words, the policy is first encouraged to converge stably under mild disturbances, and then robustness is acquired under increasingly stronger disturbances. This curriculum-based adversarial training allows the agent to be progressively exposed to more challenging environments.

The main agent outputs residual steering and residual acceleration for the same observation o_t. These two residuals are linearly combined according to the mixing coefficient η_t and then subjected to scaling and filtering (reflecting various control constraints) before being applied to the actual vehicle. In particular, the steering residual is passed through a first-order low-pass filter that blends the applied value from the previous step with the current residual candidate, thereby preventing abrupt steering-rate changes and ensuring that the same dynamic constraints used during training are also enforced during deployment.

The environment advances the vehicle model by adding the final residual to the LQR baseline input. It then returns the next observation o_{t+1}, reward r_t, constraint signals $c_j(t)$, a termination flag, and stability-related logs such as the value of the Lyapunov candidate function. The main and adversarial agents store this information in their respective replay buffers:

The main agent records the actually applied mixed residual as its action, thereby ensuring that the action representation used during training matches the input applied during execution.

Training under a min–max structure is performed by first storing the transitions collected during rollouts in a replay buffer and then repeatedly updating the parameters via mini-batch sampling. The driving policy learns an actor–critic model based on a distributional (quantile) critic and a risk-sensitive objective. In contrast, the adversarial policy is trained with TD3 using a reward whose sign is flipped from the primary reward, encouraging it to degrade the driving policy's performance.

C. CURVATURE-AWARE LONGITUDINAL SPEED CONTROL

To improve the integrated LQR–residual RL control architecture, a curvature-aware longitudinal speed control logic is introduced. This enables the vehicle to maintain the reference speed v_{ref} on straight segments while naturally decelerating in curved segments, thereby improving both ride comfort and safety. The proposed longitudinal scheme consists of three stages: (a) curvature-based dynamic reference speed generation, (b) integration with the LQR baseline and residual/adversarial acceleration inputs, and (c) overspeed protection in high-speed regions.

1) CURVATURE-BASED DYNAMIC SPEED REFERENCE

In principle, a vehicle is designed to track the user-specified reference speed v_{ref}. However, maintaining the same speed in cornering sections with large curvatures can violate lateral-acceleration limits and degrade ride comfort. Therefore, in this work, a dynamically adjusted reference speed is generated at each time step t using the road curvature κ_t. First, for the current curvature κ_t, an exponential moving average is applied to compute a noise-attenuated effective curvature $\hat{\kappa}_t$.

$$\hat{\kappa}_t = (1-\alpha_\kappa)\,\hat{\kappa}_{t-1} + \alpha_\kappa\,|\kappa_t|\,, 0 < \alpha_\kappa \le 1 \tag{42}$$

Here, α_κ is the smoothing gain used to filter the curvature signal, and sat$(\cdot)$ denotes a saturation operator enforcing comfort/safety bounds. The parameters a_{comf} and a_{sfty} specify comfort- and safety-based acceleration limits, respectively.

κ_t is used to avoids excessive sensitivity to instantaneous curvature fluctuations, while progressively capturing the curvature level during corner entry and mid-corner segments.

Next, from the perspective of lateral acceleration, we define upper bounds on the speed, satisfying comfort and safety, as outlined in (43) and (44), respectively:

$$v_{comf}\,(t) = \sqrt{\frac{a_{comf}}{\hat{\kappa}_t + \varepsilon}} \tag{43}$$

$$v_{sfty}\,(t) = \sqrt{\frac{a_{sfty}}{\hat{\kappa}_t + \varepsilon}} \tag{44}$$

where a_{comf} and a_{sfty} denote lateral-acceleration limits for ride comfort and safety, respectively; and $\varepsilon > 0$ is a small constant introduced to prevent division by zero. The curvature-based speed upper bound $v_{\text{lim}}(t)$ is then determined, together with the reference speed and physical upper/lower bounds.

$$v_{lim}\,(t) = clip\left(min\left(v_{ref}\,, v_{comf}\,(t)\,, v_{sfty}\,(t)\right), v_{min}, v_{max}\right) \tag{45}$$

On straight segments, v_{comf} and v_{sfty} are sufficiently large, and thus, $v_{\text{lim}}(t)$ approaches v_{ref}. As the curvature increases, $v_{\text{lim}}(t)$ decreases, automatically lowering the target speed in cornering sections.

The actual reference speed $v_{ref,dyn}(t)$ is implemented as a first-order dynamic system that gradually converges toward $v_{lim}(t)$. We impose an asymmetric rate limit to ensure rapid deceleration but more gradual acceleration. The rationale for designing deceleration to be faster is as follows: In corner-entry regions, the reference speed can be reduced sufficiently in advance of the curvature increase. By lowering the speed before entering the corner, overall driving safety can be enhanced.

$$\Delta v_{dec} = \gamma_{dec}\Delta t$$
$$\Delta v_{acc} = \gamma_{acc}\Delta t, (\gamma_{dec} > \gamma_{acc})$$

$$v_{ref,dyn}(t) = v_{ref,dyn}(t-\Delta t) + sat_{[-\Delta v_{dec},\Delta v_{dec}]}\left(v_{lim}(t) - v_{ref,dyn}(t-\Delta t)\right) \quad (46)$$

Here, the sat(·) operator denotes a saturation operation over the interval $[a, b]$. In other words, during corner entry, $v_{\mathrm{ref,dyn}}$ decreases rapidly to decelerate the vehicle to a curvature-appropriate speed, whereas after exiting the corner and returning to a straight segment, it accelerates more gradually to ensure ride comfort.

The generated $v_{\mathrm{ref,dyn}}(t)$ is used as the reference for defining both the velocity-error component in the LQR design state vector and longitudinal speed error in the reward function. The longitudinal speed error is defined as follows:

$$e_v(t) = v_{ref,dyn}(t) - v(t) \quad (47)$$

In this manner, longitudinal control is transformed from a fixed-speed tracking problem to a speed-profile tracking problem, with the reference automatically adjusted by curvature. Both the LQR controller and residual RL policy are then configured to be consistent with this dynamic reference.

2) INTEGRATION WITH RESIDUAL RL AND ADVERSARIAL DISTURBANCES

The dynamic longitudinal speed reference $v_{ref,dyn}(t)$ is incorporated within the integrated LQR–residual RL–adversary framework described in Section III-B2.

First, the LQR-based baseline longitudinal acceleration $a_{x,t}^{\mathrm{LQR}}$ is computed via linear quadratic regulation with respect to a state vector that includes the longitudinal states. The main agent's residual acceleration $\Delta a_{x,t}^{\mathrm{LQR}}$ and adversary's action-level and dynamics-level disturbances are then superimposed sequentially, yielding the longitudinal acceleration command inside the environment:

$$a_{x,t}^{cmd} = a_{x,t}^{LQR} + \Delta a_{x,t}^{RL} + b_{a_x,t}^{adv} \quad (48)$$

Here, $b_{a_x,t}^{\mathrm{adv}}$ represents the contributions of the adversary's outputs, including the longitudinal-acceleration bias and friction-coefficient scaling.

Notably, even when using the curvature-based speed reference, the actual speed v_t may exceed $v_{ref,dyn}(t)$ owing to disturbances during driving or an unstable policy in the early stages of training. To prevent this, the proposed algorithm introduces an overspeed guard, which automatically limits the acceleration command when

$$v_t > v_{ref,dyn}(t) + \varepsilon_v \Longrightarrow \begin{cases} a_{x,t}^{cmd} \le 0 \\ a_{x,t}^{cmd} \leftarrow a_{x,t}^{cmd} - k_{ov}\left(v(t) - v_{ref,dyn}(t)\right) \end{cases} \quad (49)$$

Here, ε_v denotes the allowable speed margin, and $k_{\mathrm{ov}} > 0$ is a feedback gain that increases deceleration in proportion to the degree of overspeed. This logic ensures that, even if the main agent outputs an excessively positive residual acceleration or the adversary manipulates the friction scaling, situations in which the vehicle persistently violates the curvature-based speed limit are naturally suppressed.

Overall, by integrating the curvature-dependent longitudinal speed logic into the LQR–residual RL–adversary unified framework, the proposed algorithm achieves stable and robust lane-keeping and speed-tracking performance even under diverse road-curvature patterns and disturbance conditions.

V. EXPERIMENTS AND RESULTS

This section describes the simulation environment, training and test road scenarios, and experimental results.

A. SIMULATION ENVIRONMENT AND ROAD SCENARIOS

Experiments are conducted on a lane-keeping scenario implemented in the CARLA simulator (v0.10.0, Windows 64-bit Shipping) in synchronous mode with a fixed simulation time step of $\Delta t = 0.01$s. At each control step, the client advances the world by one tick and applies the computed control, ensuring deterministic and reproducible rollouts. The controller does not use camera/LiDAR perception. Instead, it uses simulator-provided ground-truth ego states (pose and velocity from the CARLA API) to compute tracking errors with respect to the reference path. The road environment is generated from an ASAM OpenDRIVE (.xodr) file exported from IPG CarMaker (OpenDRIVE 1.7). This OpenDRIVE-based road definition, together with the simulator-provided vehicle states, enables controlled road configurations and supports reproducible evaluation of the integrated LQR–residual RL control architecture and adversarial RL framework described in Section III.

The training road is an S-shaped track, as shown in Fig. 5. The trained agent is evaluated on four test roads: T-shaped, O-shaped, U-shaped, and random-shaped, as illustrated in Fig. 6.

The baseline methods used for comparison are RL_PP_PID [8], MPC, and LQR.

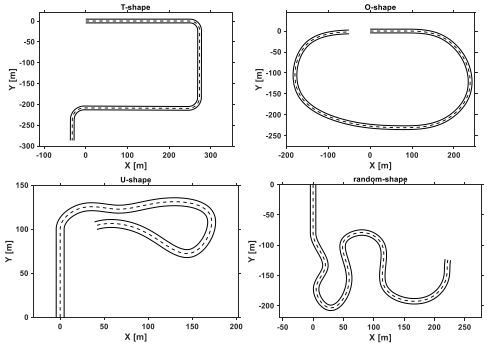


FIGURE 5. Test road.

LQR is a representative linear optimal control method based on a linearized vehicle model and quadratic cost function. It is a widely used model-based baseline controller for lane keeping and path-tracking problems [49]. Moreover, as LQR serves as the baseline controller in the proposed

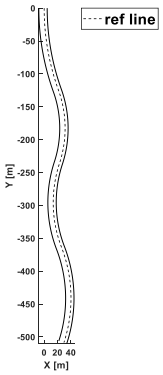


FIGURE 6. S-shaped road.

integrated architecture, this analysis enables a quantitative evaluation of how the proposed algorithm improves tracking performance and robustness relative to LQR under the same model and weighting settings.

MPC is a representative predictive control approach that can explicitly incorporate constraints such as steering and acceleration limits, jerk, and lane-departure conditions, while using the same vehicle model and prediction horizon. In autonomous driving, MPC has been widely adopted as a high-performance baseline for lane keeping, lane changes, and adaptive cruise control [50], [51]. In this study, as constraints are incorporated in the learning component and the same vehicle model and prediction horizon are used, comparison with MPC enables the quantitative assessment of the extent to which the proposed algorithm improves tracking performance.

B. EVALUATION METHODOLOGY

This section describes the experimental procedure used to evaluate the proposed integrated LQR–residual RL–adversarial RL controller. All baseline controllers are evaluated under the same CARLA simulation environment and the same road scenarios, and performance is compared based on statistics computed over the full driving segment of each episode.

To evaluate robustness, evaluations are performed with the trained adversarial policy activated. The adversarial agent takes the same observation as the main agent and outputs a multidimensional disturbance vector consisting of action, dynamics, and observation disturbances. Among these, the action-disturbance components are mixed with the main agent's residual steering and acceleration inputs. The mixing ratio is fixed to maintain a disturbance level comparable to that during training. All disturbances are saturated within predefined upper and lower bounds to prevent the vehicle from entering non-physical states.

To further address potential policy dependence of learned-adversary stress tests, we additionally report trajectory overlays under the proposed random 12D disturbance setting. As shown in Fig. 10, the proposed controller maintains stable tracking behavior across all test roads under randomized perturbations sampled within the same bounds.

The remaining disturbance components are injected within the environment to perturb the vehicle's acceleration/steering response, road friction, sensor bias and noise, observation delays, and related factors, thereby emulating scenarios where model mismatch and sensor uncertainty occur simultaneously.

The evaluation metrics are organized into three categories: tracking performance, ride comfort, and control stability. Tracking performance is quantified using the root mean square error (RMSE) of the lateral error $|\ e_y\ |$ and longitudinal speed error $|\ e_v\ |$ with respect to the reference trajectory. Ride comfort is assessed using the mean absolute value of the lateral jerk, numerically estimated from the vehicle's lateral acceleration. Notably, a larger jerk indicates more uncomfortable abrupt acceleration/deceleration and steering.

We additionally compare the CVaR-based objective with a risk-neutral mean-Q objective and evaluate the effect of the constraint-handling term under the same quantile-critic and residual RL-LQR structure. The detailed results are reported in Tables 12 and 13.

In addition to the main comparisons, we conduct targeted ablation studies under an identical evaluation protocol to clarify the contribution of key components. Specifically, we compare the proposed distributional (quantile) critic with a standard TD3 critic while keeping the residual RL–LQR control structure and all other settings unchanged. We also evaluate the effect of the curvature-aware longitudinal speed logic by toggling it on/off for the quantile-critic controller, to verify that this module serves as an auxiliary heuristic rather than a principal contributor of the learning component. Along with tracking and comfort metrics (e.g., RMSE/peak errors and jerk), we explicitly report safety/constraint-related outcomes including the collision rate, the off-road rate, and the route-violation rate, where the off-road and route-violation rates are defined as the fraction of time steps satisfying $|e_y| \geq 3$ m and $|e_y| \geq 2$ m, respectively.

To quantify control stability, we define a Lyapunov candidate function $V = x^\top P x$ based on the state vector used in the LQR design [52] and compute its temporal change over an episode as $\Delta V(t) = V(t) - V(t - \Delta t)$. Here, $x = [e_y, v_y, e_\psi, e_r, e_v, e_{y,\text{int}}]^\top$, and P is the solution of the Riccati equation. We use the Lyapunov candidate V and the derived quantity $m(t)$ as a Lyapunov-like monitoring signal for energy-growth regulation, rather than as a formal stability certificate for the residual-augmented closed loop. In this context, $m(t) > 0$ is treated as an empirical violation of the decrease condition.

$$m(t) \triangleq \Delta V(t) + \eta \|x(t)\|^2 - \beta \leq 0 \tag{50}$$

We use the Lyapunov candidate $V(x) = x^\top P x$ to compute $\Delta V(t) = V(t) - V(t - \Delta t)$, and monitor the Lyapunov-metric $m(t)$ defined in (50) as an empirical signal rather than a formal stability certificate for the residual-augmented closed loop. The threshold β in (50) is calibrated from the first N_{calib}

samples of $\Delta V(t)+\eta \parallel x(t) \parallel^2$ collected under the LQR-only closed loop while keeping the learned adversarial disturbance injection enabled, using $\beta = \text{Quantile}_q(\Delta V(t) + \eta \parallel x(t) \parallel^2) + \beta_{\text{margin}}$. During evaluation, we report the distribution of $m(t)$ and the violation rate $\mathbb{P}(m(t) > 0)$.

The quantile-based threshold β is pre-estimated from closed-loop data of the standalone LQR controller under the same adversarial disturbance conditions and then fixed for evaluation. Using the same β, we compare the distribution of $m(t)$ and the violation indicator $m(t) > 0$ to assess whether the residual input increases Lyapunov-metric violations [53].

Calibrating β from LQR-only rollouts provides a conservative reference envelope for the Lyapunov-metric $m(t)$ under the same environment configuration, enabling a fair comparison of how residual learning affects the monitored violations. When the disturbance regime, speed profile, or road conditions change substantially, β should be re-calibrated; in this work we keep the configuration fixed per experiment and discuss sensitivity to β as a limitation.

This methodology enables the comparison of the proposed integrated LQR–residual RL–adversarial RL controller against conventional LQR, MPC, and RL_PP_PID controllers using a consistent benchmark, under both nominal conditions (without disturbances) and conditions with injected adversarial disturbances.

C. EXPERIMENTAL RESULTS

To address the stochasticity inherent in RL, we train the proposed method with three random seeds (0, 1, 2) and report the mean and standard deviation. During evaluation, we use a deterministic actor by setting the action exploration noise to zero, and we fix the scenario and disturbance sampling seeds for reproducibility. This deterministic (noise-free) evaluation is consistent with typical deployment for stable execution, but may underrepresent real-world variability; we therefore also report results under explicit and randomized disturbances. Fig. 7 shows the learning curves aggregated across seeds for the proposed min–max adversarial training. In the early stage, the main-agent return exhibits noticeable oscillations that gradually diminish; it then increases and reaches a stable plateau, suggesting improved closed-loop tracking under the learned disturbance regime. Conversely, the adversary return evolves in the opposite direction and stabilizes, indicating progressively more challenging perturbations within the bounded disturbance set. Overall, the reduced fluctuation and stabilized trends provide empirical evidence of stable, non-divergent two-player training convergence.

In addition, we compare the critic loss formulations (Huber vs. MSE) to provide stability-related evidence. Since the two objectives have different scales, we interpret the curves in terms of spike magnitude and variance rather than absolute values. As shown in Fig. 8, the critic losses exhibit similar overall convergence trends, while the quantile-Huber critic produces fewer and smaller spike events and a lower variability than the MSE-based critic. Since the two objectives are not directly comparable in absolute scale, we interpret this result in terms of training stability, i.e., reduced sensitivity to occasional large-magnitude TD errors that can arise under the disturbance-driven non-stationary training regime. We then examine control stability using the Lyapunov-metric residual $m(t)$.

Fig. 9 shows the profile of the residual $m(t)$ over time, used as a Lyapunov-like monitoring signal for empirical violations of the decrease condition. The proposed method maintains $m(t)$ at a level comparable to that of the LQR baseline, and no increase is observed in either the frequency or magnitude of positive residuals ($m(t) > 0$), attributable to the addition of the residual input. Therefore, the proposed method maintains a comparable Lyapunov-metric profile to the LQR baseline, as reflected by similar (or reduced) $m(t)$ violation frequency and magnitude under the tested conditions. We further discuss the sensitivity to β by considering (re-)estimation of β under different conditions (e.g., LQR-only vs. adversary-on) and its impact on the reported $m(t)$ violation rates.

Next, we examine the qualitative characteristics of the proposed RL–LQR integrated controller through a set of visualizations based on the abovementioned evaluation metrics. Fig. 10 further reports the trajectory overlays under a policy-agnostic random 12-dimensional disturbance injection, where each disturbance channel is sampled within the same bounds as Table 1. The results under the disturbance-free and disturbance-containing conditions are shown in Figs. 11 and 12, respectively.

In the nominal environment without disturbances, all controllers operate based on the nominal model, ensuring basic stability and tracking performance. For LQR, as $P > 0$ is designed to satisfy the Lyapunov decrease condition $\Delta V \leq -\alpha \parallel x_k \parallel^2$ for $A_{\text{cl}} = A + BK$, stability is theoretically guaranteed under the nominal model, and the controller exhibits rapid convergence for small errors. Similarly, because MPC's predictions based on the nominal system $(A \mid B)$ match the true dynamics in this setting, the optimized state and input sequences can be implemented accurately, yielding excellent tracking performance without constraint violations or prediction errors.

The proposed RL–LQR residual architecture enables fine-grained adjustments even in this nominal regime while preserving LQR-based stability. In particular, by leveraging error terms in the reward function, prediction errors, and related signals, the policy can learn subtle refinements beyond what a fixed-gain LQR can provide. This includes compensating for nonlinear response effects and optimally rebalancing control effort in response to speed–curvature interactions. Consequently, the proposed approach achieves the lowest tracking errors even in the nominal environment. Overall, in the absence of disturbances, the proposed algorithm outperforms LQR, MPC, and RL_PP_PID, indicating that the RL-based residual performs effective fine-level optimization even under nominal conditions.

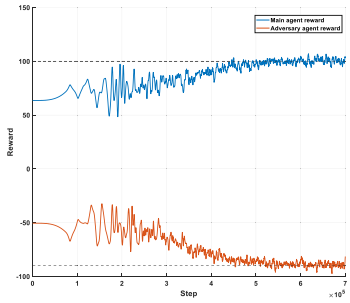


FIGURE 7. 3 seeds average reward plot.

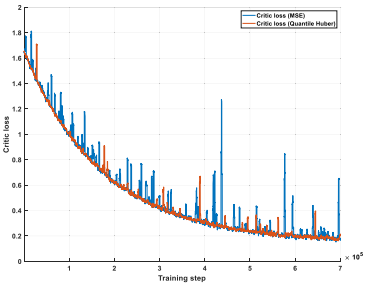


FIGURE 8. MSE critic loss & quantile huber critic loss.

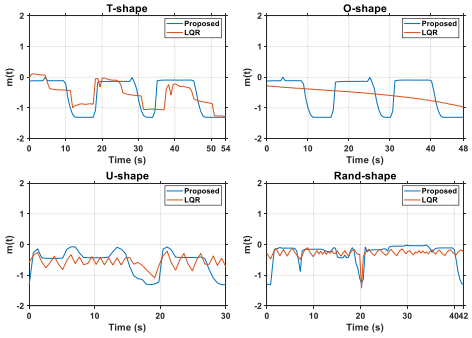


FIGURE 9. Profile of the Lyapunov-metric residual $m(t)$ over time, used to monitor empirical violations of the decrease condition.

The RL_PP_PID method in [10] essentially combines geometric control (PP) with error-feedback control (PID). Consequently, it cannot fully leverage the structural advantages of model-based optimal feedback when the nominal model is accurate. In contrast, the proposed RL–LQR residual architecture preserves the stability of LQR in the nominal regime, while using the residual to compensate for nonlinear errors in curvature-varying segments, thereby achieving smaller peak errors and lower cumulative errors than [10].

Tables 6 and 7 summarize the RMSE and peak error of lateral tracking error e_y and longitudinal speed error e_v, respectively, under the disturbance-free (nominal) condition. All RMSE and peak-error values reported in the tables are averaged over three independently trained runs with random seeds 0, 1, and 2.

Under conditions involve disturbances, the proposed algorithm demonstrates superior tracking performance compared with the other controllers across all road scenarios. From a control-stability perspective, in the presence of disturbances, the Lyapunov decrease property guaranteed by LQR under the nominal model can be violated more readily. In the nominal closed loop, $P > 0$ is chosen such that, for $A_{\mathrm{cl}} = A + BK$, the decrease condition $\Delta V \leq -\alpha \parallel x_k \parallel^2$ is satisfied. However, in the actual system, uncertainties ΔA, ΔB, and disturbances w_k are present, effectively perturbing the closed-loop dynamics to $A_{\mathrm{cl}} + \Delta_{\mathrm{cl}}$. In this case, the Lyapunov function increment $\Delta V(x_k)$ is influenced simultaneously by the term $(A_{\mathrm{cl}} + \Delta_{\mathrm{cl}})^\top P(A_{\mathrm{cl}} + \Delta_{\mathrm{cl}}) - P$ and by disturbance-related terms. When Δ_{cl} exceeds a threshold, the corresponding matrix may no longer be negative definite, implying that there exist states for which $\Delta V > 0$. In this scenario, outside the nominal regime (i.e., under large disturbances and model mismatch), LQR can lose its energy-decrease property, leading to performance degradation in the form of accumulated or even diverging tracking errors.

Similarly, MPC predicted the trajectory $\hat{x}_{k+i}$ based on the nominal model $(A \mid B)$ and minimizes a cost function; however, when the true dynamics are affected by $(A + \Delta A, B + \Delta B)$ and an external disturbance w, the prediction error $x_{k+i} - \hat{x}_{k+i}$ increases. As this mismatch grows, state/input constraints that appear satisfied in the prediction may be violated in the real system, and the optimization based on nominal predictions can no longer guarantee optimality, stability, or constraint satisfaction in the actual dynamics. This may potentially lead to a sharp degradation in tracking performance.

RL_PP_PID [10] does not explicitly model disturbances or partially unknown dynamics. Its design focuses on the resulting increase in tracking error and smoothness of the control input. This structure can respond by increasing the PID contribution when disturbances enlarge the tracking error, thereby reducing the error more rapidly. However, under large disturbances or significant model mismatch, more aggressive steering may be selected to suppress the error, which increases input variability—i.e., jerk—and can lead to stability issues.

In contrast, the proposed LQR–residual architecture builds on the baseline stabilizing property of LQR while learning a residual input that directly suppresses energy increases in regions where disturbances have a strong effect, achieving improved robustness. Specifically, the reward function includes a constraint term that penalizes violations of the stability condition $\Delta V \leq \eta \| x \|^2 + \beta$, where β is interpreted as an upper bound on the allowable energy increase under disturbances. As a result, even if ΔV cannot always be made negative in the presence of disturbances, the policy is encouraged to keep the energy increase within the range specified by β, preventing the state trajectory from diverging beyond the nominal regime and maintaining it within a bounded magnitude.

Moreover, policy optimization incorporates the Lyapunov-constraint violation into the objective and minimizes it jointly with a Lagrange multiplier, while a

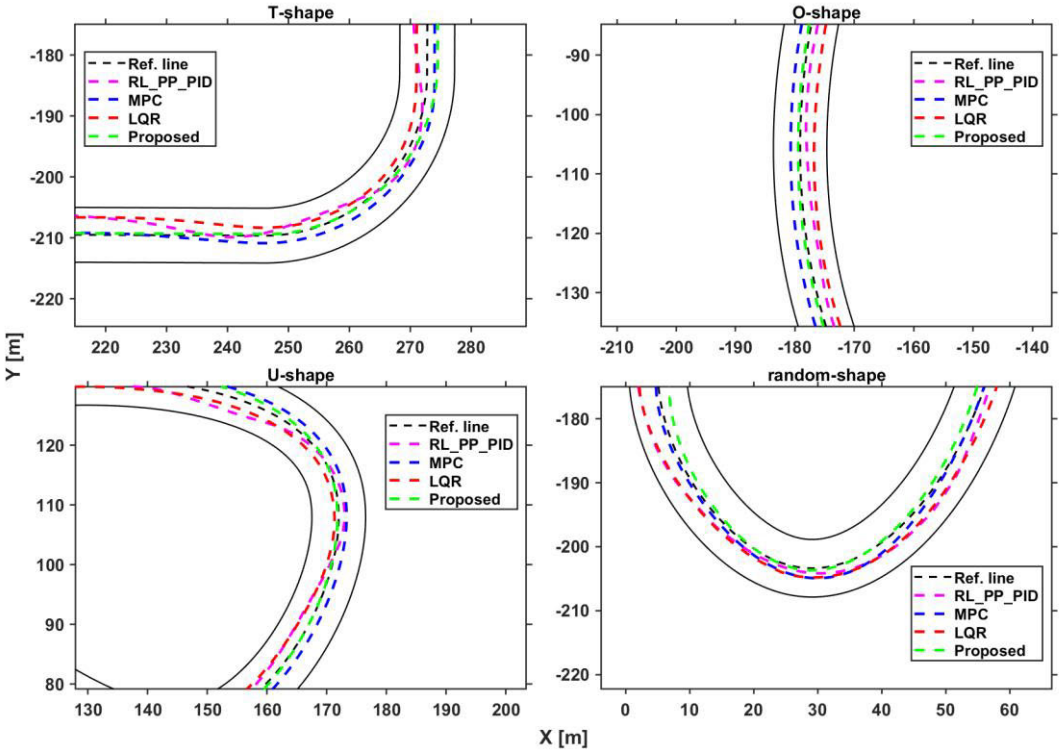


FIGURE 10. Trajectory overlays under policy-agnostic random 12-dimensional disturbance injection across the test roads.

TABLE 2. Ey RMSE/peak error for each test road (random disturbance).

	S_shape (training)	T_shape	O_shape	U_shape	Rand_shape
RL_PP_PID	0.7426±0.0660/3.2918	1.6428±0.1420/3.0415	1.7339±0.1510/2.9586	1.1734±0.1020/3.0197	1.3114±0.1140/3.0472
MPC	1.2035±0.1040/3.3276	1.9184±0.1670/3.2461	1.1327±0.0990/3.1628	2.0715±0.1810/3.3382	1.5876±0.1380/2.9413
LQR	1.9628±0.1780/3.9014	2.0219±0.1840/3.8067	2.2984±0.2100/3.5486	2.7423±0.2460/3.4418	2.0612±0.1880/3.6427
Proposed	0.0368±0.0033/0.1394	0.0627±0.0056/0.0918	0.0843±0.0074/0.1412	0.0786±0.0069/0.1631	0.0664±0.0059/0.1649

TABLE 3. Ev RMSE/peak error for each test road (random disturbance).

	S_shape (training)	T_shape	O_shape	U_shape	Rand_shape
RL_PP_PID	0.8517±0.0750/1.7629	0.6814±0.0610/2.7683	0.4726±0.0430/1.8791	1.1986±0.1060/1.9687	0.8588±0.0760/1.9326
MPC	1.8426±0.1660/2.4928	1.8824±0.1710/2.6849	1.8715±0.1690/2.7684	1.6912±0.1530/2.1965	1.8841±0.1710/2.7017
LQR	1.8325±0.1650/2.9286	1.7368±0.1570/2.9075	1.7361±0.1570/2.7842	1.7489±0.1580/2.6214	1.7427±0.1580/2.9183
Proposed	0.4681±0.0420/1.1937	0.2036±0.0180/1.0984	0.1898±0.0170/0.9986	0.4418±0.0390/0.9917	0.4429±0.0390/0.9828

TABLE 4. Quantile vs normal Ey RMSE/peak error/route-violation rate.

	S_shape (training)	T_shape	O_shape	U_shape	Rand_shape
Normal critic	0.0489/0.1637/0%	0.0794/0.1148/0%	0.1046/0.1679/0%	0.0988/0.1916/0%	0.0847/0.1895/0%
Quantile critic (Proposed)	**0.0417/0.1492/0%**	**0.0701/0.1003/0%**	**0.0922/0.1515/0%**	**0.0870/0.1742/0%**	**0.0731/0.1728/0%**

TABLE 5. Lon speed logic Ey Rmse/peak error/ route-violation rate.

	S_shape (training)	T_shape	O_shape	U_shape	Rand_shape
OFF	0.0483/0.1618/0%	0.0756/0.1139/0%	0.1038/0.1687/0%	0.0979/0.1898/0%	0.0852/0.1874/0%
ON (Proposed)	**0.0417/0.1492/0%**	**0.0701/0.1003/0%**	**0.0922/0.1515/0%**	**0.0870/0.1742/0%**	**0.0731/0.1728/0%**

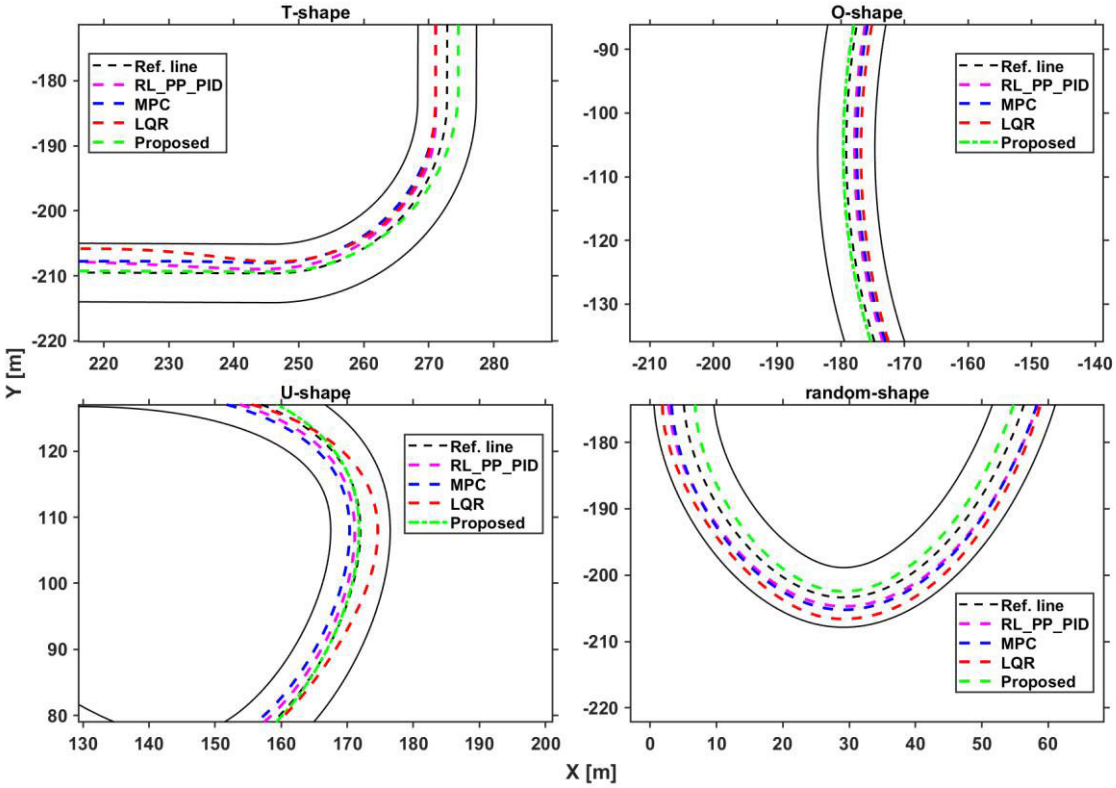


FIGURE 11. Results under disturbance-free conditions.

TABLE 6. Ey RMSE/peak error for each test road (no disturbance).

	S_shape(training)	T_shape	O_shape	U_shape	Rand_shape
RL_PP_PID	0.0881±0.0065/0.2267	0.0144±0.0000/0.2530	0.0510±0.0000/0.3808	0.0428±0.0000/0.3724	0.0840±0.0000/0.4127
MPC	0.0912±0.0072/0.2452	0.0417±0.0038/0.1315	0.0720±0.0058/0.1050	0.0720±0.0061/0.1892	0.0872±0.0071/0.1243
LQR	0.6520±0.0580/1.6650	0.8070±0.0760/2.1120	0.3040±0.0300/0.6530	0.6810±0.0630/1.4010	0.8420±0.0780/2.4050
Proposed	**0.0438±0.0040/0.1001**	**0.0118±0.0010/0.0464**	**0.0227±0.0021/0.0710**	**0.0412±0.0036/0.1487**	**0.0596±0.0051/0.1664**

CVaR-based risk-sensitive objective further emphasizes performance in worst-case segments. Consequently, the proposed RL-based residual controller naturally adjusts the control action to reduce ΔV when constraints are violated, mitigating tracking-performance degradation even under disturbances.

Tables 9 and 10 present the RMSE and peak error of e_y and e_v, respectively, under the disturbance-containing conditions.

As shown in Fig. 10, the proposed controller maintains stable tracking under policy-agnostic random disturbances, despite the broader variability introduced by random sampling. Unlike the learned-adversary case, the random disturbances are not optimized to maximize tracking degradation; nevertheless, they provide a controller-independent stress test within the same disturbance envelope (Table 1). The consistent qualitative trend supports that the robustness of the

TABLE 7. Ev RMSE/peak error for each test road (no disturbance).

	S_shape(training)	T_shape	O_shape	U_shape	Rand_shape
RL_PP_PID	0.4285±0.0420/1.9120	0.4860±0.0480/1.9600	0.3230±0.0320/0.5210	0.7120±0.0710/1.9620	0.4890±0.0480/1.9750
MPC	1.2680±0.1240/1.9580	1.3920±0.1360/1.8660	1.4710±0.1450/1.9440	0.5920±0.0590/1.6240	1.0040±0.1000/1.8330
LQR	1.7060±0.1680/2.0410	1.5280±0.1510/1.9520	1.9870±0.1950/1.8220	1.2540±0.1240/2.0050	0.9920±0.0980/1.9720
Proposed	**0.3590±0.0340/1.0340**	**0.2240±0.0220/1.0280**	**0.2140±0.0210/0.4630**	**0.4980±0.0470/1.3420**	**0.4860±0.0460/0.9580**

TABLE 8. Lateral jerk for each test road (no disturbance).

	S_shape (training)	T_shape	O_shape	U_shape	Rand_shape
RL_PP_PID	0.0668	0.0134	0.0280	0.0289	0.0429
MPC	0.0889	0.0857	0.0583	0.0546	0.0682
LQR	0.0946	0.0836	0.0797	0.0728	0.0851
Proposed	**0.0326**	**0.0404**	**0.0333**	**0.0268**	**0.0181**

TABLE 9. Ey RMSE/peak error for each test road (disturbance).

	S_shape(training)	T_shape	O_shape	U_shape	Rand_shape
RL_PP_PID	0.9820±0.0820/3.4140	1.7040±0.1410/3.1730	1.8120±0.1510/3.0840	1.2380±0.1030/3.1440	1.3820±0.1150/3.1900
MPC	1.2680±0.1040/3.4470	2.0060±0.1680/3.3460	1.1950±0.1010/3.2650	2.1620±0.1810/3.4680	1.6620±0.1370/3.0410
LQR	2.0480±0.1780/4.0610	2.1060±0.1830/3.9560	2.4010±0.2110/3.6940	2.8420±0.2420/3.5790	2.1420±0.1870/3.7580
Proposed	**0.0423±0.0038/0.1526**	**0.0714±0.0062/0.1024**	**0.0936±0.0081/0.1544**	**0.0883±0.0076/0.1768**	**0.0744±0.0064/0.1751**

TABLE 10. Ev RMSE/peak error for each test road (disturbance).

	S_shape (training)	T_shape	O_shape	U_shape	Rand_shape
RL_PP_PID	0.8920±0.0810/1.8340	0.7120±0.0660/2.8620	0.5050±0.0470/1.9450	1.2520±0.1160/2.0610	0.9010±0.0830/2.0110
MPC	1.9220±0.1760/2.5830	1.9360±0.1780/2.7750	1.9370±0.1770/2.8440	1.7460±0.1600/2.2760	1.9460±0.1790/2.7810
LQR	1.9060±0.1750/3.0140	1.7960±0.1650/2.9960	1.8020±0.1660/2.8580	1.8120±0.1670/2.7030	1.8250±0.1680/3.0120
Proposed	**0.4910±0.0450/1.2470**	**0.2140±0.0200/1.1450**	**0.2010±0.0190/1.0340**	**0.4660±0.0430/1.0390**	**0.4680±0.0430/1.1260**

TABLE 11. Lateral jerk for each test road (disturbance).

	S_shape (training)	T_shape	O_shape	U_shape	Rand_shape
RL_PP_PID	0.8934	0.7081	0.8672	0.9006	0.9193
MPC	0.9122	0.9340	0.8447	0.7910	0.8949
LQR	0.9643	0.9582	0.8691	0.9544	1.0028
Proposed	**0.0739**	**0.0608**	**0.0436**	**0.0501**	**0.0541**

proposed method is not limited to a specific learned adversarial policy and generalizes to randomized perturbations.

Tables 8 and 11 report the jerk of each controller across the different road scenarios under the disturbance-free and disturbance-containing conditions, respectively. As the proposed algorithm imposes a penalty on jerk increases in the reward function, the policy converges toward avoiding excessive acceleration/deceleration and abrupt steering changes.

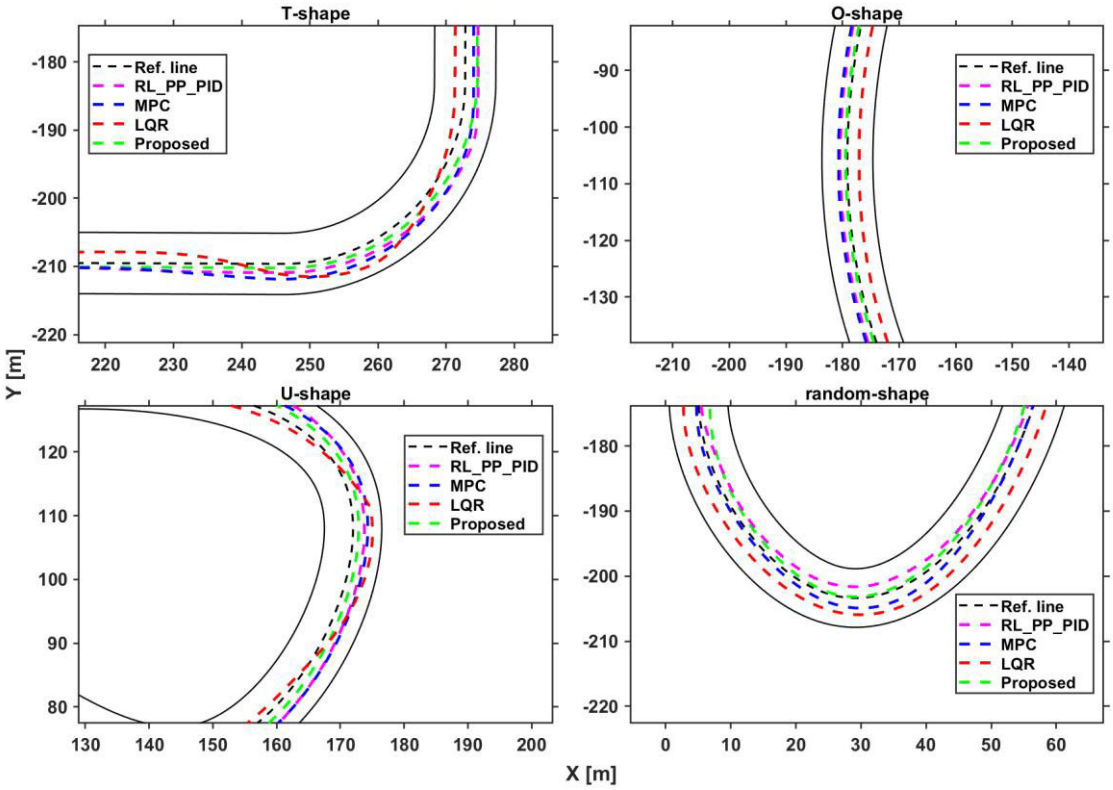


FIGURE 12. Results under involving disturbance.

TABLE 12. Cvar vs normal(mean-Q) Ey RMSE/peak error.

	S_shape (training)	T_shape	O_shape	U_shape	Rand_shape
Normal(mean-Q)	0.0478/0.1609	0.0793/0.1127	0.0998/0.1638	0.0942/0.1836	0.0816/0.1798
CVaR	**0.0417/0.1492**	**0.0701/0.1003**	**0.0922/0.1515**	**0.0870/0.1742**	**0.0731/0.1728**

TABLE 13. Constraint on/off Ey RMSE/peak error.

	S_shape (training)	T_shape	O_shape	U_shape	Rand_shape
OFF	0.0497/0.1623	0.0794/0.1142	0.1056/0.1659	0.0981/0.1854	0.0897/0.1816
ON	**0.0417/0.1492**	**0.0701/0.1003**	**0.0922/0.1515**	**0.0870/0.1742**	**0.0731/0.1728**

Owing to this structural property, the temporal variation of the control inputs is naturally suppressed not only in the nominal setting but also under disturbances, resulting in consistently lower jerk values.

The ablation comparing the proposed distributional (quantile) critic against the standard TD3 critic, with the residual RL_LQR structure and all other settings held fixed, shows that the quantile-critic controller yields more stable learning and consistently better tracking performance. In particular, it reduces both the e_y RMSE and peak lateral errors while maintaining comparable or improved comfort (jerk) levels, and it also improves safety/constraint-related outcomes (lower off-road and route-violation rates). These results indicate that the performance gains are supported by multiple components of the proposed learning design. Since our experiments do not consider surrounding traffic, no collisions occurred; accordingly, the collision rate was 0% in all evaluations.

Table 12 shows that the CVaR-based objective yields lower lateral RMSE and peak lateral error than the mean-Q objective, supporting the contribution of the risk-sensitive objective.

Table 14 shows that enabling the constraint-handling term improves lateral RMSE and peak lateral error across all test

roads, supporting its role in improved tracking performance under disturbances.

The comparison with and without the curvature-aware longitudinal speed logic shows that the tracking and comfort metrics (e.g., RMSE/peak errors and jerk) are only marginally affected by including this module. This indicates that the performance improvements primarily stem from the learned residual RL_LQR component and its core learning design, while the curvature-aware speed logic functions as a complementary, safety-oriented heuristic. The results are shown in Table 5.

VI. CONCLUSION

This study proposes an integrated control architecture for CARLA-based path and speed tracking, combining an LQR-based baseline controller with a learned residual control input, while simultaneously injecting adversarial disturbances during training to enhance robustness under disturbances. The proposed method preserves the fundamental stabilizing properties of LQR, while enabling the residual policy to correct errors in regions with significant model mismatch and sensor uncertainty.

Experimental results show that the proposed algorithm significantly reduces tracking errors compared with the baseline controllers (LQR, MPC, and RL_PP_PID) under both disturbance-free and disturbance-containing conditions. In particular, in the presence of disturbances, LQR and MPC—which rely on nominal model assumptions—exhibit performance degradation owing to mismatches between the nominal and true dynamics. In contrast, the proposed method maintains robust tracking performance by leveraging the learned residual input to counteract the effects of disturbances.

To quantitatively assess closed-loop stability when the residual input is added on top of LQR, we introduce a stability monitor based on the Lyapunov candidate function $V = x^\top Px$ derived from the LQR design. Under the same adversarial disturbance conditions, we first pre-estimated and fix β using closed-loop driving data from the standalone LQR controller. When evaluating the residual policy, violations of the condition $\Delta V \leq \eta \parallel x \parallel^2 + \beta$ are compared using the same β. The proposed method maintains bounded Lyapunov-metric behavior under the tested conditions, as evidenced by the reported $m(t)$ violation statistics. We emphasize that these results provide empirical evidence of robustness and ultimate-boundedness indicators, rather than a formal closed-loop stability proof for the residual-augmented system. This indicates that the proposed residual policy not only reduces tracking errors but also regulates energy growth within a bounded upper limit, even under disturbances.

Future work can be aimed at extended the training scenarios and refining the generalization methodology to ensure robust tracking performance on more diverse road geometries, where longitudinal grade (e.g., uphill and downhill) and curvature vary jointly. While this focuses on learning a residual input, the proposed framework can be extended to an end-to-end control architecture that directly learns control commands without a baseline controller, or to a scheme that adaptively determines the mixing ratio with the baseline controller. Lastly, for real-vehicle deployment, a systematic expansion of safety assurance procedures is required, including more faithful modeling that reflects practical uncertainties—such as sensor noise, parameter variations, and changes in road friction—together with formal verification and safety monitoring.

APPENDIX A
RL TRAINING HYPERPARAMETERS

See Table 14.

TABLE 14. Training parameters.

Symbol	Quantity
Total steps	700,000 steps
Horizon	8,192 steps
Update-to-data ratio	1 update per 16 env steps (512 updates per 8,192 steps)
Batch size	256
Discount factor (γ)	0.99
Soft target update (τ)	0.005
Learning rate	3e-4
Policy delay	2
Target policy smoothing noise	*target_noise* = 0.2, *noise_clip* = 0.5
Actor network (hidden units)	(256, 256)
Critic network (hidden units)	(256, 256)
Number of quantiles	25
Quantile Huber $\rho_\tau^{\delta H}$	1.0
CVaR level (α)	0.2
Replay buffer size	560,000
Replay buffer size (adv.)	16,384 (= 2 × 8,192)
Exploration noise (adv.)	0.0
Min-max mixing weight (η)	0.00 (<50k), 0.3 (<150k), 0.45 (<300k), 0.50 (otherwise)

APPENDIX B
LQR BASELINE PARAMETERS

See Table 15.

TABLE 15. LQR parameters.

Symbol	Quantity
Sampling time	0.01s
Linearization speed	linearized at $Vx \approx 9.72$ m/s
State vector	$[e_y, v_y, e_\psi, e_r, e_v, e_{y,\text{int}}]^\top$
Weight matrices	Q = diag(1,1,1,1,1,1), R = diag(1,1)
Vehicle mass	1800 kg
Yaw inertia	2750 kg·m²
CG to front axle	1.23 m
CG to rear axle	1.62 m
Front cornering stiffness	200000 N/rad
Rear cornering stiffness	175000 N/rad

APPENDIX C
MPC CONTROLLER PARAMETERS

See Table 16.

TABLE 16. MPC parameters.

Symbol	Quantity
Sampling time	0.01s
Prediction horizon length	60 steps
Total prediction time	0.6s
State vector	$[e_y, v_y, e_\psi]$
Control input	$[\delta, a]$
Cost weight (lateral error)	$e_y^2 : 0.9$
Cost weight (heading error)	$e_y^2 : 1.0$
Cost weight (speed tracking)	$(v - v_{ref})^2 : 0.5$
Cost weight (steering effort)	δ^2: 1.0
Cost weight (acceleration effort)	$a^2 : 1.0$
Cost weight (steering rate)	$(\Delta\delta)^2 : 1.0$
Cost weight (acceleration rate)	$(\Delta a)^2 : 1.0$
Input bound (steering)	$\lvert\delta\rvert \leq 0.55$
Input bound (acceleration)	$-5.5 \leq a \leq 3.2$
Input rate bound (steering)	$\lvert\delta_k - \delta_{k-1}\rvert \leq 0.18$
Input rate bound (acceleration)	$\lvert a_k - a_{k-1}\rvert \leq 0.50$
Solver	IPOPT (Interior-Point OPTimizer)

REFERENCES

[1] D. Zeng, S. Pan, Y. Yu, Y. Hu, J. Yang, P. Zhang, L. Xiong, G. Carbone, and L. Gao, "A comparative study on trajectory tracking control methods for automated vehicles," *Sci. Rep.*, vol. 15, no. 1, May 2025, Art. no. 17073, doi: 10.1038/s41598-025-01365-9.

[2] Y. Hu, M. Wu, J. Kang, and R. Yu, "D-tracking: Digital twin enabled trajectory tracking system of autonomous vehicles," *IEEE Trans. Veh. Technol.*, vol. 73, no. 11, pp. 16457–16469, Nov. 2024, doi: 10.1109/TVT.2024.3414410.

[3] S. Xu, H. Peng, and Y. Tang, "Preview path tracking control with delay compensation for autonomous vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 5, pp. 2979–2989, May 2021, doi: 10.1109/TITS.2020.2978417.

[4] Z.-A. Zheng, Z. Ye, and X. Zheng, "Intelligent vehicle lateral control strategy research based on feedforward + predictive LQR algorithm with GA optimisation and PID compensation," *Sci. Rep.*, vol. 14, no. 1, Sep. 2024, Art. no. 22317, doi: 10.1038/s41598-024-72960-5.

[5] Q. Yao, Y. Shi, P. Hang, and Y. Tian, "Path tracking robust control strategy for intelligent vehicle based on force-driven with MPC and H_∞," *Chin. J. Mech. Eng.*, vol. 38, no. 1, Sep. 2025, Art. no. 184, doi: 10.1186/s10033-025-01294-8.

[6] K. Geng and S. Liu, "Robust path tracking control for autonomous vehicle based on a novel fault tolerant adaptive model predictive control algorithm," *Appl. Sci.*, vol. 10, no. 18, p. 6249, Sep. 2020, doi: 10.3390/app10186249.

[7] X. He and C. Lv, "Towards safe autonomous driving: Decision making with observation-robust reinforcement learning," *Automot. Innov.*, vol. 6, no. 4, pp. 509–520, Nov. 2023, doi: 10.1007/s42154-023-00256-x.

[8] I. Batkovic, A. Gupta, M. Zanon, and P. Falcone, "Experimental validation of safe MPC for autonomous driving in uncertain environments," *IEEE Trans. Control Syst. Technol.*, vol. 31, no. 5, pp. 2027–2042, Sep. 2023, doi: 10.1109/TCST.2023.3291533.

[9] N. A. Spielberg, M. Brown, and J. C. Gerdes, "Neural network model predictive motion control applied to automated driving with unknown friction," *IEEE Trans. Control Syst. Technol.*, vol. 30, no. 5, pp. 1934–1945, Sep. 2022, doi: 10.1109/TCST.2021.3130225.

[10] Y. Shan, B. Zheng, L. Chen, L. Chen, and D. Chen, "A reinforcement learning-based adaptive path tracking approach for autonomous driving," *IEEE Trans. Veh. Technol.*, vol. 69, no. 10, pp. 10581–10595, Oct. 2020, doi: 10.1109/TVT.2020.3014628.

[11] M. Fliess and C. Join, "Model-free control," *Int. J. Control*, vol. 86, no. 12, pp. 2228–2252, Dec. 2013, doi: 10.1080/00207179.2013.810345.

[12] Y. Yin, Z. Xu, Y. Wei, W. Jiang, D. Yao, and Z. Dong, "Model-free sliding mode control for path following in autonomous vehicle," *J. Franklin Inst.*, vol. 362, no. 12, Aug. 2025, Art. no. 107806, doi: 10.1016/j.jfranklin.2025.107806.

[13] J. Han, "From PID to active disturbance rejection control," *IEEE Trans. Ind. Electron.*, vol. 56, no. 3, pp. 900–906, Mar. 2009, doi: 10.1109/TIE.2008.2011621.

[14] A. Baciu and C. Lazar, "Iterative feedback tuning of model-free intelligent PID controllers," *Actuators*, vol. 12, no. 2, p. 56, Jan. 2023, doi: 10.3390/act12020056.

[15] M. Li, R. Kang, D. T. Branson, and J. S. Dai, "Model-free control for continuum robots based on an adaptive Kalman filter," *IEEE/ASME Trans. Mechatronics*, vol. 23, no. 1, pp. 286–297, Feb. 2018, doi: 10.1109/TMECH.2017.2775663.

[16] R. Marino, S. Scalzi, and M. Netto, "Nested PID steering control for lane keeping in autonomous vehicles," *Control Eng. Pract.*, vol. 19, no. 12, pp. 1459–1467, Dec. 2011, doi: 10.1016/j.conengprac.2011.08.005.

[17] R. P. Borase, D. K. Maghade, S. Y. Sondkar, and S. N. Pawar, "A review of PID control, tuning methods and applications," *Int. J. Dyn. Control*, vol. 9, no. 2, pp. 818–827, Jun. 2021, doi: 10.1007/s40435-020-00665-4.

[18] Y. Huang, Z. Tian, Q. Jiang, and J. Xu, "Path tracking based on improved pure pursuit model and PID," in *Proc. IEEE 2nd Int. Conf. Civil Aviation Saf. Inf. Technol. (ICCASIT*, Weihai, China, Oct. 2020, pp. 359–364, doi: 10.1109/ICCASIT50869.2020.9368694.

[19] B. Paden, M. Cáp, S. Z. Yong, D. Yershov, and E. Frazzoli, "A survey of motion planning and control techniques for self-driving urban vehicles," *IEEE Trans. Intell. Vehicles*, vol. 1, no. 1, pp. 33–55, Mar. 2016, doi: 10.1109/TIV.2016.2578706.

[20] P. Falcone, F. Borrelli, J. Asgari, H. E. Tseng, and D. Hrovat, "Predictive active steering control for autonomous vehicle systems," *IEEE Trans. Control Syst. Technol.*, vol. 15, no. 3, pp. 566–580, May 2007, doi: 10.1109/TCST.2007.894653.

[21] J. Su, J. Wu, P. Cheng, and J. Chen, "Autonomous vehicle control through the dynamics and controller learning," *IEEE Trans. Veh. Technol.*, vol. 67, no. 7, pp. 5650–5657, Jul. 2018, doi: 10.1109/TVT.2018.2819806.

[22] Z. Wang, K. Sun, S. Ma, L. Sun, W. Gao, and Z. Dong, "Improved linear quadratic regulator lateral path tracking approach based on a real-time updated algorithm with fuzzy control and cosine similarity for autonomous vehicles," *Electronics*, vol. 11, no. 22, p. 3703, Nov. 2022, doi: 10.3390/electronics11223703.

[23] C. Piao, X. Liu, and C. Lu, "Lateral control using parameter self-tuning LQR on autonomous vehicle," in *Proc. Int. Conf. Intell. Comput., Autom. Syst. (ICICAS)*, Chongqing, China, Dec. 2019, pp. 913–917, doi: 10.1109/ICICAS48597.2019.00197.

[24] M. Park and Y. Kang, "Experimental verification of a drift controller for autonomous vehicle tracking: A circular trajectory using LQR method," *Int. J. Control, Autom. Syst.*, vol. 19, no. 1, pp. 404–416, Jan. 2021, doi: 10.1007/s12555-019-0757-2.

[25] S. Mata, A. Zubizarreta, and C. Pinto, "Robust tube-based model predictive control for lateral path tracking," *IEEE Trans. Intell. Vehicles*, vol. 4, no. 4, pp. 569–577, Dec. 2019, doi: 10.1109/TIV.2019.2938102.

[26] H. Wang, B. Liu, X. Ping, and Q. An, "Path tracking control for autonomous vehicles based on an improved MPC," *IEEE Access*, vol. 7, pp. 161064–161073, 2019, doi: 10.1109/ACCESS.2019.2944894.

[27] S.-P. Chen, G.-M. Xiong, H.-Y. Chen, and D. Negrut, "MPC-based path tracking with PID speed control for high-speed autonomous vehicles considering time-optimal travel," *J. Cent. South Univ.*, vol. 27, no. 12, pp. 3702–3720, Dec. 2020, doi: 10.1007/s11771-020-4561-1.

[28] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, "Constrained model predictive control: Stability and optimality," *Automatica*, vol. 36, no. 6, pp. 789–814, Jun. 2000, doi: 10.1016/s0005-1098(99)00214-9.

[29] D. Kamran, J. Zhu, and M. Lauer, "Learning path tracking for real car-like mobile robots from simulation," in *Proc. Eur. Conf. Mobile Robots (ECMR)*, Prague, Czech Republic, Sep. 2019, pp. 1–6, doi: 10.1109/ECMR.2019.8870947.

[30] K. Alomari, R. Mendoza, D. Goehring, and R. Rojas, "Path following with deep reinforcement learning for autonomous cars," in *Proc. 2nd Int. Conf. Robot., Comput. Vis. Intell. Syst.*, 2021, pp. 173–181, doi: 10.5220/0010715400003061.

[31] A. V. Carrasco and J. S. Sequeira, "Tuning path tracking controllers for autonomous cars using reinforcement learning," *PeerJ Comput. Sci.*, vol. 9, p. e1550, Oct. 2023, doi: 10.7717/peerj-cs.1550.

[32] A. Joglekar, S. Sathe, N. Misurati, S. Srinivasan, M. J. Schmid, and V. Krovi, "Deep reinforcement learning based adaptation of pure-pursuit path-tracking control for skid-steered vehicles," *IFAC-PapersOnLine*, vol. 55, no. 37, pp. 400–407, Jan. 2022, doi: 10.1016/j.ifacol.2022.11.216.

[33] A. Roy, H. Xu, and S. Pokutta, "Reinforcement learning under model mismatch," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 3043–3052.

[34] J. Moos, K. Hansel, H. Abdulsamad, S. Stark, D. Clever, and J. Peters, "Robust reinforcement learning: A review of foundations and recent advances," *Mach. Learn. Knowl. Extraction*, vol. 4, no. 1, pp. 276–315, Mar. 2022, doi: 10.3390/make4010013.
[35] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proc. 31st Int. Conf. Mach. Learn. (ICML)*, Beijing, China, Jun. 2014, pp. 387–395.
[36] S. Fujimoto, H. V. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, Stockholm, Sweden, Jul. 2018, pp. 1587–1596, doi: 10.48550/arxiv.1802.09477.
[37] T. Xu, Z. Meng, W. Lu, and Z. Tong, "End-to-end autonomous driving decision method based on improved TD3 algorithm in complex scenarios," *Sensors*, vol. 24, no. 15, Jul. 2024, Art. no. 4962, doi: 10.3390/s24154962.
[38] A. A. Al-Atawi, "Genetically optimized TD3 algorithm for efficient access control in the Internet of Vehicles," *Wireless Netw.*, vol. 30, no. 9, pp. 7581–7601, Apr. 2024, doi: 10.1007/s11276-024-03733-1.
[39] L. Gisslen, A. Eakins, C. Gordillo, J. Bergdahl, and K. Tollmar, "Adversarial reinforcement learning for procedural content generation," in *Proc. IEEE Conf. Games*, Aug. 2021, pp. 1–8, doi: 10.1109/COG52621.2021.9619053.
[40] X. He, B. Lou, H. Yang, and C. Lv, "Robust decision making for autonomous vehicles at highway on-ramps: A constrained adversarial reinforcement learning approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 4103–4113, Apr. 2023, doi: 10.1109/TITS.2022.3229518.
[41] S. Gupta, G. Singal, D. Garg, and S. Jagannathan, "QC_SANE: Robust control in DRL using quantile critic with spiking actor and normalized ensemble," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 9, pp. 6656–6662, Sep. 2023, doi: 10.1109/TNNLS.2021.3129525.
[42] W. Dabney, G. Ostrovski, D. Silver, and R. Munos, "Implicit quantile networks for distributional reinforcement learning," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, Stockholm, Sweden, Jan. 2018, pp. 1096–1105. Accessed: Jan. 14, 2026. [Online]. Available: https://proceedings.mlr.press/v80/dabney18a.html
[43] D. Gupta, B. B. Hazarika, and M. Berlin, "Robust regularized extreme learning machine with asymmetric Huber loss function," *Neural Comput. Appl.*, vol. 32, no. 16, pp. 12971–12998, Aug. 2020, doi: 10.1007/s00521-020-04741-w.
[44] P. Zhao, F. Yu, and Z. Wan, "A Huber loss minimization approach to Byzantine robust federated learning," in *Proc. AAAI Conf. Artif. Intell.*, Mar. 2024, pp. 21806–21814, doi: 10.1609/aaai.v38i19.30181.
[45] Z. Huang, J. Zhang, R. Tian, and Y. Zhang, "End-to-end autonomous driving decision based on deep reinforcement learning," in *Proc. 5th Int. Conf. Control, Autom. Robot. (ICCAR)*, Beijing, China, Apr. 2019, pp. 658–662, doi: 10.1109/ICCAR.2019.8813431.
[46] J. Chen, S. E. Li, and M. Tomizuka, "Interpretable end-to-end urban autonomous driving with latent deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 6, pp. 5068–5078, Jun. 2022, doi: 10.1109/TITS.2020.3046646.
[47] T. Staessens, T. Lefebvre, and G. Crevecoeur, "Adaptive control of a mechatronic system using constrained residual reinforcement learning," *IEEE Trans. Ind. Electron.*, vol. 69, no. 10, pp. 10447–10456, Oct. 2022, doi: 10.1109/TIE.2022.3144565.
[48] M. Rokonuzzaman, N. Mohajer, S. Nahavandi, and S. Mohamed, "Review and performance evaluation of path tracking controllers of autonomous vehicles," *IET Intell. Transp. Syst.*, vol. 15, no. 5, pp. 646–670, May 2021, doi: 10.1049/itr2.12051.
[49] P. Stano, U. Montanaro, D. Tavernini, M. Tufo, G. Fiengo, L. Novella, and A. Sorniotti, "Model predictive path tracking control for automated road vehicles: A review," *Annu. Rev. Control*, vol. 55, pp. 194–236, Jan. 2023, doi: 10.1016/j.arcontrol.2022.11.001.
[50] L. Guo, P. Ge, D. Sun, and Y. Qiao, "Adaptive cruise control based on model predictive control with constraints softening," *Appl. Sci.*, vol. 10, no. 5, p. 1635, Feb. 2020, doi: 10.3390/app10051635.
[51] F. L. Lewis and D. Vrabie, "Reinforcement learning and adaptive dynamic programming for feedback control," *IEEE Circuits Syst. Mag.*, vol. 9, no. 3, pp. 32–50, Sep. 2009, doi: 10.1109/MCAS.2009.933854.
[52] Y. Chow, O. Nachum, A. Faust, E. Duenez-Guzman, and M. Ghavamzadeh, "Lyapunov-based safe policy optimization for continuous control," 2019, *arXiv:1901.10031*.
[53] T. J. Perkins and A. G. Barto, "Lyapunov design for safe reinforcement learning," *J. Mach. Learn. Res.*, vol. 3, pp. 803–832, Dec. 2003.



DONGHYUN LEE received the B.S. degree in mechanical engineering from Kookmin University, Seoul, Republic of Korea, in 2024, where he is currently pursuing the M.S. degree in automotive mobility with the Graduate School of Automotive Engineering.

His research interests include reinforcement learning-based decision and control algorithms, as well as the development and refinement of reinforcement learning-based vehicle localization methods.



TAEYOUNG OH received the B.S. degree in electrical engineering from Pai Chai University, Daejeon, Republic of Korea, in 2019. He is currently pursuing the Ph.D. degree in automotive engineering with Kookmin University, Seoul, Republic of Korea. His research interests include vehicle dynamics, planning and control for autonomous vehicles, and autonomous driving simulations.



JINKYEOM KIM received the B.S. degree in mechatronics engineering from the Tech University of Korea, Siheung, Republic of Korea, in 2026. He is currently pursuing the M.S. degree in automotive engineering with Kookmin University, Seoul, Republic of Korea.

His research interests include vehicle dynamics, decision and control for autonomous driving, and autonomous driving simulation.



JINWOO YOO (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electrical engineering from Pohang University of Science and Technology (POSTECH), in 2009, 2011, and 2015, respectively. He was a Senior Engineer at Samsung Electronics, from 2015 to 2019. He is currently an Associate Professor with the Department of Automobile and IT Convergence, College of Automotive Engineering, Kookmin University. His research interests include autonomous driving technologies and signal/image processing techniques.